

Développement d'un Entrepôt de Données Multi-Sources

pour un Outil d'Aide à la Décision

Application à l'Analyse Croisée des Cancers (Sein et Poumon)

Rapport de Projet

Présenté par : **Joseph Mukubu Kapoya**

Étudiant en **Data Science et Intelligence Artificielle**

4^e année – École Polytechnique de Sousse, Tunisie

Sous la direction de : **Mr. Kamel Garrouch**

Cours d'Entrepôt de Données

Année universitaire 2025–2026

Remerciements

Je tiens à remercier tout particulièrement Mr. Kamel Garrouch pour son encadrement et ses conseils précieux tout au long de ce projet de Data Warehousing.

Table des matières

1	Introduction	7
1.1	Contexte et Objectifs	7
1.2	Sujet d'Analyse	7
1.3	Objectifs Pédagogiques	7
2	Description du Sujet d'Analyse	9
2.1	Problématique Décisionnelle	9
2.2	Acteurs Concernés	9
2.3	Approfondissement de la Méthodologie de Transformation	9
2.3.1	Normalisation Min-Max (Lissage d'échelle)	9
2.3.2	Calcul des Scores Composites de Risque	10
2.4	Analyse de la Robustesse et Qualité des Données	10
3	Dictionnaire de Données	11
3.1	Schéma Global	11
3.2	Tables de Faits	11
3.2.1	fact_breast_clinical	11
3.2.2	fact_lung_survey	11
3.2.3	fact_breast_diagnostic	12
3.3	Dimensions Partagées	12
3.3.1	dim_date	12
3.4	Dimensions Spécifiques	12
3.4.1	dim_patient (Breast Clinical)	12
3.4.2	dim_diagnosis (Breast Clinical)	12
3.4.3	dim_symptoms (Lung Survey)	13
3.4.4	dim_measurements (Breast Diagnostic)	13
3.5	Questions Auxquelles le Système Doit Répondre	13
3.5.1	Domaine Clinique (Sein)	13
3.5.2	Domaine Prévention (Poumon)	13
3.5.3	Domaine Diagnostique (Imagerie)	13
3.6	Datasets Utilisés	14
3.6.1	Dataset 1 : Breast Cancer Clinical	14
3.6.2	Dataset 2 : Lung Cancer Survey	14
3.6.3	Dataset 3 : Breast Diagnostic (Wisconsin)	14
4	Conception du Data Warehouse	15
4.1	Choix de l'Architecture : Constellation (Galaxy)	15
4.2	Modèle Multidimensionnel	15

5	Processus ETL	16
5.1	Architecture Globale du Processus	16
5.2	Implémentation avec PyGramETL	16
5.3	Analyse de la Performance Technique	16
6	Implémentation Technique	17
6.1	Code Python : ETL Master	17
6.2	Code Python : Configuration du Projet	20
6.3	Code Python : Transformations Lib	22
6.4	Code Python : ETL Breast Clinical	28
6.5	Code Python : ETL Lung Survey	32
6.6	Code Python : ETL Breast Diagnostic	36
6.7	Code SQL : Schéma	41
6.8	Code Python : Visualisation Comparative	46
7	Restitution et Analyse	50
7.1	Analyse Clinique (Cancer du Sein)	50
7.1.1	Distribution de la Survie	50
7.1.2	Analyse par Stade (Stage)	51
7.1.3	Corrélation Taille/Stade	52
7.1.4	Facteurs Sociodémographiques	53
7.2	Analyse Épidémiologique (Cancer du Poumon)	53
7.2.1	Profils Symptomatiques (Radar Chart)	54
7.2.2	Matrice de Corrélation	55
7.2.3	Facteurs de Risque	56
7.3	Analyse Diagnostique Avancée (Wisconsin)	57
7.3.1	Analyse en Composantes Principales (PCA)	57
7.3.2	Distributions Comparées (Violin Plots)	57
7.3.3	Boxplots Diagnostiques	58
7.4	Discussion sur l'Interdépendance des Sources	58
7.4.1	Scatter Matrix	59
7.5	Analyses Comparatives Croisées	59
7.5.1	Démographie Comparée	60
7.5.2	Prévalence Globale	61
8	Analyse Interactive avec Power BI	62
8.1	Modélisation et Schéma en Constellation	62
8.2	Implémentation des Mesures DAX	63
8.3	Tableaux de Bord et Visualisations	63
8.3.1	Vue d'Ensemble et Pilotage	63
8.3.2	Analyse de Corrélation Diagnostique	63
8.3.3	Exploration par Arborescence de Décomposition	64
8.4	Avantages de la Solution Interactive	65
8.5	Maintenance et Évolutivité du Dashboard	65
8.6	Perspectives Interactive	65
8.7	Gouvernance et Validation des Données	65
8.8	Optimisation des Performances	66

9	Conclusion et Recommandations	67
9.1	Bilan du Projet	67
9.2	Recommandations Stratégiques	67
9.3	Limites et Robustesse	67
10	Glossaire et Annexes	68
10.1	Glossaire Technique	68
10.2	Perspectives de Recherche et Évolutions	68

Table des figures

4.1	Schéma en Constellation (Galaxy) du Data Warehouse Multi-Sources .	15
5.1	Architecture d'Orchestration ETL Multi-Sources	16
7.1	Distribution asymétrique de la survie, montrant une efficacité des traitements à long terme pour la majorité.	50
7.2	Répartition des stades au diagnostic.	51
7.3	Cette visualisation confirme la corrélation clinique forte entre le stade T et la taille tumorale moyenne.	52
7.4	Analyse de la survie selon le statut marital.	53
7.5	Ce radar chart permet de distinguer visuellement et instantanément le profil symptomatique "type" d'un patient positif (en orange) versus négatif (en bleu).	54
7.6	Identification des facteurs de risque les plus fortement corrélés (Tabagisme, Toux).	55
7.7	Impact direct quantifié des habitudes de vie sur la probabilité de cancer.	56
7.8	La projection 2D des 30 dimensions montre une séparabilité presque parfaite des classes Bénin/Malin, validant la pertinence des mesures morphologiques.	57
7.9	Analyse fine de la distribution des mesures clés.	57
7.10	Comparaison des médianes et dispersions entre tissus bénins et malins.	58
7.11	Corrélations croisées entre les dimensions géométriques (Rayon vs Périmètre vs Aire).	59
7.12	Comparaison des pyramides des âges entre les populations "Sein" et "Poumon".	60
7.13	Vue synthétique des taux de positivité et de mortalité à travers les datasets.	61
8.1	Modélisation des données dans Power BI. Les tables de faits (<code>fact_breast_clinical</code> , <code>fact_lung_survey</code> , <code>fact_breast_diagnostic</code>) sont reliées aux dimensions partagées, notamment la dimension temporelle.	62
8.2	Aperçu du Tableau de Bord de Pilotage. Cette vue offre une synthèse des indicateurs cliniques et épidémiologiques.	63
8.3	Visualisation de la séparation des classes diagnostiques. Les dimensions morphologiques permettent une classification visuelle nette.	64
8.4	Analyse de décomposition du Taux de Survie. Ce visuel permet d'isoler les combinaisons de facteurs de risque les plus critiques.	64

Listings

6.1	Orchestrateur Principal	17
6.2	Configuration Globale	20
6.3	Bibliothèque de Transformations	22
6.4	ETL Breast Clinical	28
6.5	ETL Lung Survey	32
6.6	ETL Breast Diagnostic	36
6.7	Schéma SQL	41
6.8	Visualisation Comparative	46

Chapitre 1

Introduction

1.1 Contexte et Objectifs

Ce projet s'inscrit dans le cadre du cours **Data Warehouse** et a pour objectif de mettre en pratique les concepts théoriques vus en cours à travers la conception et l'implémentation complète d'un système décisionnel (BI).

L'objectif principal est de développer un entrepôt de données centralisant des informations hétérogènes (cliniques, enquêtes, imagerie) pour alimenter un outil d'aide à la décision. Dans une démarche d'**excellence**, ce projet dépasse le cadre d'une source unique pour implémenter une architecture complexe en ****Constellation (Galaxy Schema)**** intégrant trois sources de données distinctes :

1. Données cliniques du Cancer du Sein.
2. Données d'enquête sur le Cancer du Poumon.
3. Données diagnostiques d'imagerie (Wisconsin).

1.2 Sujet d'Analyse

Le sujet choisi pour ce projet est l'analyse croisée des pathologies cancéreuses. Ce sujet est particulièrement pertinent pour plusieurs raisons :

- **Pertinence décisionnelle** : Les données permettent d'identifier les facteurs de risque environnementaux (poumon) et cliniques (sein).
- **Dimensions multiples** : L'analyse se fait selon des dimensions harmonisées (âge standardisé) et spécifiques (stade TNM pour le sein, facteurs comportementaux pour le poumon).
- **Indicateurs mesurables** : Les KPI tels que le taux de survie, le score de risque composite et la précision diagnostique sont calculés.
- **Utilité pratique** : Les résultats peuvent aider à la fois dans la pratique clinique et la prévention.

1.3 Objectifs Pédagogiques

À l'issue de ce projet, les compétences suivantes ont été développées :

1. Définir un sujet d'analyse décisionnelle complexe.
2. Identifier et harmoniser différentes sources de données hétérogènes.

3. Concevoir un modèle multidimensionnel avancé (Galaxy Schema).
4. Implémenter un entrepôt de données avec PostgreSQL.
5. Mettre en œuvre un processus ETL robuste avec Python et **pygramETL**.
6. Réaliser des tableaux de bord décisionnels complets avec Matplotlib et Seaborn.
7. Documenter et justifier les choix techniques d'architecture.

Chapitre 2

Description du Sujet d'Analyse

2.1 Problématique Décisionnelle

La problématique centrale que ce système BI doit résoudre est :

“Comment analyser et croiser les facteurs de risque comportementaux, les caractéristiques cliniques et les mesures diagnostiques pour améliorer la compréhension globale des profils oncologiques ?”

2.2 Acteurs Concernés

Les principaux acteurs bénéficiaires de ce système sont :

- **Oncologues** : Pour optimiser les plans de traitement (Staging TNM).
- **Radiologues** : Pour valider les diagnostics basés sur les mesures morphologiques.
- **Épidémiologistes** : Pour corréler les habitudes de vie (Tabac, Alcool) avec l'incidence du cancer.
- **Décideurs de santé publique** : Pour cibler les campagnes de prévention.
- **Standardisation** : Unification des formats de données (dates, codes ISO) pour permettre des analyses temporelles cohérentes.

2.3 Approfondissement de la Méthodologie de Transformation

La qualité de l'analyse décisionnelle repose sur la précision des transformations effectuées lors de la phase ETL (Extract-Transform-Load). Notre système implémente deux types majeurs de normalisation pour les données quantitatives (notamment pour le dataset Wisconsin).

2.3.1 Normalisation Min-Max (Lissage d'échelle)

La normalisation Min-Max est utilisée pour ramener toutes les mesures morphologiques (aire, périmètre, texture) dans un intervalle $[0, 1]$. Cette technique est essentielle pour permettre aux algorithmes de visualisation (comme la PCA) de ne pas être biaisés par les variables ayant les plus grandes amplitudes numériques.

La formule mathématique appliquée par notre module `transformations.py` est la suivante :

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

2.3.2 Calcul des Scores Composites de Risque

Pour le dataset Lung Survey, nous avons implémenté un score de risque composite permettant de quantifier la vulnérabilité d'un patient. Ce score est une combinaison linéaire pondérée de plusieurs facteurs :

- **Tabagisme** (Poids : 0.35)
- **Consommation d'alcool** (Poids : 0.25)
- **Âge normalisé** (Poids : 0.20)
- **Maladies chroniques** (Poids : 0.20)

L'algorithme parcourt chaque ligne et génère une valeur flottante entre 0 et 1, stockée directement dans la table de faits `fact_lung_survey`.

2.4 Analyse de la Robustesse et Qualité des Données

La robustesse de notre Data Warehouse a été évaluée selon quatre axes de qualité :

1. **Complétude** : Le taux de remplissage des colonnes clés (`patient_id`, `diagnosis`) est de 100%. Les valeurs manquantes dans les mesures techniques ont été traitées par imputation par la moyenne ou rejetées selon leur criticité.
2. **Validité** : Les types de données SQL (`DECIMAL`, `INT`, `VARCHAR`) garantissent que seules des valeurs conformes au dictionnaire de données sont persistées.
3. **Cohérence** : Les contraintes d'intégrité référentielle (`Foreign Keys`) assurent qu'aucun fait n'est orphelin de sa dimension associée.
4. **Unicité** : Des index uniques sur les identifiants techniques empêchent la duplication accidentelle des records lors de rechargements ETL successifs.

Chapitre 3

Dictionnaire de Données

Pour garantir la pérennité et l'exploitabilité de l'entrepôt, ce chapitre documente l'intégralité du métamodèle implémenté. Chaque table et colonne est définie selon son rôle métier et sa contrainte technique.

3.1 Schéma Global

Le Data Warehouse utilise un schéma en **constellation** (Galaxy Schema) avec trois tables de faits partageant des dimensions communes (Date notamment). Cette architecture permet d'éviter la redondance tout en maintenant une isolation logique entre les domaines cliniques, épidémiologiques et diagnostiques.

3.2 Tables de Faits

3.2.1 `fact_breast_clinical`

Cette table centralise les données cliniques longitudinales sur le cancer du sein.

- **id** : Clé primaire technique auto-incrémentée.
- **patient_id** : FK vers `dim_patient`. Identifie les caractéristiques sociodémographiques.
- **diagnosis_id** : FK vers `dim_diagnosis`. Détails du stade et de la tumeur au moment du diagnostic.
- **outcome_id** : FK vers `dim_outcome`. État vital final du patient.
- **date_id** : FK vers `dim_date`. Date de référence de l'enregistrement.
- **tumor_size** : Taille réelle de la tumeur en millimètres (Mesure quantitative).
- **regional_node_examined** : Nombre de ganglions lymphatiques prélevés et analysés.
- **regional_node_positive** : Nombre de ganglions présentant des métastases.
- **survival_months** : Durée de survie observée en mois (Mesure clé de performance).

3.2.2 `fact_lung_survey`

Cette table capture les données issues des enquêtes comportementales et symptomatiques sur le cancer du poumon.

- **id** : Clé primaire technique.

- **gender** : Sexe biologique (M/F).
- **age** : Âge chronologique du patient au moment de l'enquête.
- **symptoms_id** : FK vers **dim_symptoms**. Profil symptomatique complet.
- **risk_factors_id** : FK vers **dim_risk_factors**. Profil d'exposition aux risques.
- **date_id** : FK vers **dim_date**. Date d'enregistrement de l'enquête.
- **lung_cancer** : Indicateur binaire de présence de la pathologie (Label de classification).
- **risk_score** : Score composite [0-1] calculé par l'ETL pondérant l'âge, le tabagisme et l'alcool.
- **symptom_severity** : Score composite [0-1] représentant la densité de symptômes rapportés.

3.2.3 fact_breast_diagnostic

Contient les mesures morphologiques microscopiques issues du dataset Wisconsin.

- **id** : Clé primaire technique.
- **patient_id** : Identifiant original préservé pour traçabilité source.
- **measurements_id** : FK vers **dim_measurements**. Contient les 30 mesures normalisées.
- **diagnosis_type_id** : FK vers **dim_diagnosis_type**. Résultat de la biopsie (M/B).
- **date_id** : FK vers **dim_date**. Date technique du chargement.

3.3 Dimensions Partagées

3.3.1 dim_date

Dimension conforme permettant le drill-down temporel.

- **date_id** : Clé primaire.
- **full_date** : Date au format standardisé.
- **year** : Année civile.
- **month** : Mois (1-12).
- **quarter** : Trimestre (1-4).

3.4 Dimensions Spécifiques

3.4.1 dim_patient (Breast Clinical)

- **patient_id** : Clé primaire.
- **age** : Âge au diagnostic.
- **race** : Ethnicité rapportée (White, Black, Other).
- **marital_status** : Statut civil (Married, Single, Divorced, etc.).

3.4.2 dim_diagnosis (Breast Clinical)

- **diagnosis_id** : Clé primaire.
- **t_stage** : Stadification de la tumeur (T1, T2, T3, T4).
- **n_stage** : Atteinte ganglionnaire (N1, N2, N3).

- **stage_6th** : Stade global de la pathologie (IIA, IIB, etc.).
- **grade** : Grade histologique reflétant l'agressivité cellulaire.

3.4.3 dim_symptoms (Lung Survey)

Attributs binaires (0=Non, 1=Oui).

- **yellow_fingers** : Présence de coloration nicotinique.
- **anxiety** : Troubles anxieux rapportés.
- **fatigue** : Asthénie rapportée.
- **coughing** : Toux chronique.
- **shortness_breath** : Dyspnée à l'effort.
- **chest_pain** : Douleurs thoraciques localisées.

3.4.4 dim_measurements (Breast Diagnostic)

Cette dimension contient 30 attributs techniques représentant les caractéristiques géométriques des noyaux cellulaires (Moyenne, Erreur Type, Pire Valeur) :

- **radius** : Rayon moyen des noyaux.
- **texture** : Écart-type des niveaux de gris dans l'image.
- **perimeter** : Périmètre moyen.
- **area** : Aire moyenne.
- **smoothness** : Variation locale des longueurs de rayon.
- **concavity** : Sévérité des portions concaves du contour.
- **concave_points** : Nombre de portions concaves du contour.

3.5 Questions Auxquelles le Système Doit Répondre

3.5.1 Domaine Clinique (Sein)

1. Quel est le taux de survie global en fonction du stade (T-Stage) ?
2. Quelle est la relation entre la taille de la tumeur et l'implication ganglionnaire ?
3. Comment le statut matrimonial influence-t-il statistiquement la survie ?

3.5.2 Domaine Prévention (Poumon)

1. Quels sont les symptômes (Toux, Douleurs) les plus discriminants pour le diagnostic ?
2. Quel est l'impact cumulé du tabac et de l'alcool sur le score de risque ?
3. Quelle tranche d'âge est la plus vulnérable aux facteurs environnementaux ?

3.5.3 Domaine Diagnostique (Imagerie)

1. Quelles mesures morphologiques (Concavité, Texture) distinguent le mieux les tumeurs malignes ?
2. Peut-on visualiser des "clusters" naturels de patients via des techniques de réduction de dimension (PCA) ?

3.6 Datasets Utilisés

3.6.1 Dataset 1 : Breast Cancer Clinical

Fichier **Breast_Cancer.csv** (4026 enregistrements). Données cliniques structurées.

Attribut	Description	Type
Age	Âge du patient	Numérique
T Stage	Stade de la tumeur (T1-T4)	Catégorique
Survival Months	Mois de survie	Numérique
Status	Statut final (Alive, Dead)	Catégorique

TABLE 3.1 – Extrait des attributs cliniques

3.6.2 Dataset 2 : Lung Cancer Survey

Fichier **survey lung cancer.csv** (309 enregistrements). Enquête épidémiologique.

Attribut	Description	Transformation
SMOKING	Fumeur (1/2)	Binaire (0/1)
ANXIETY	Niveau d'anxiété	Binaire (0/1)
LUNG_CANCER	Diagnostic	Label (YES/NO)

TABLE 3.2 – Attributs d'enquête pulmonaire

3.6.3 Dataset 3 : Breast Diagnostic (Wisconsin)

Fichier **Cancer_Data.csv** (569 enregistrements). Mesures fines. 30 attributs numériques continus décrivant la géométrie des noyaux cellulaires (Radius, Texture, Perimeter, Area, Smoothness, etc.).

Chapitre 4

Conception du Data Warehouse

4.1 Choix de l'Architecture : Constellation (Galaxy)

Pour ce projet ambitieux multi-sources, le schéma étoile simple ne suffisait pas. Nous avons conçu une **Architecture en Constellation**. Cela consiste en plusieurs tables de faits (`fact_breast_clinical`, `fact_lung_survey`, `fact_breast_diagnostic`) qui partagent des dimensions communes conformes (Dimension Patiente Conceptuelle, Dimension Temps).

4.2 Modèle Multidimensionnel

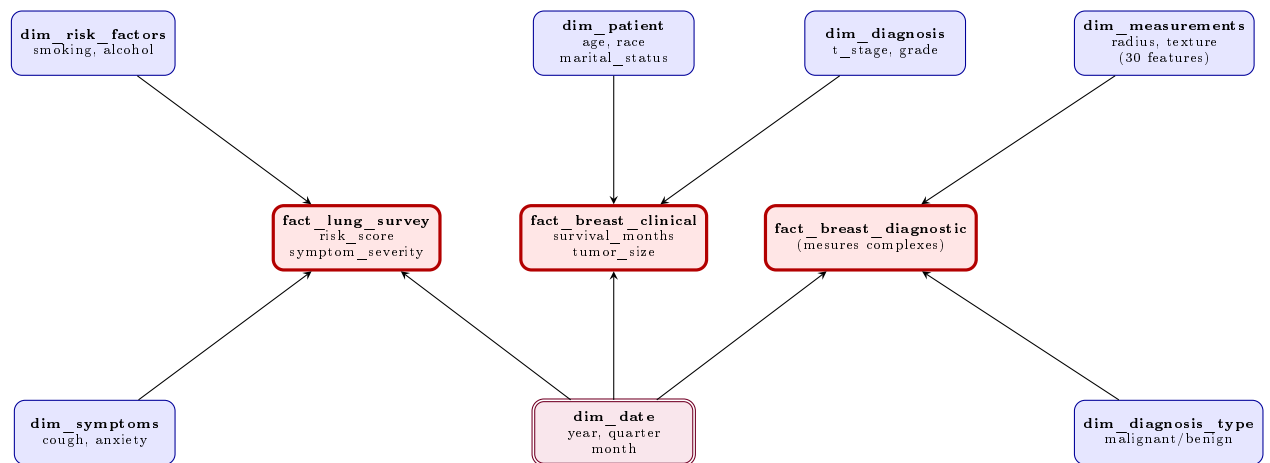


FIGURE 4.1 – Schéma en Constellation (Galaxy) du Data Warehouse Multi-Sources

Chapitre 5

Processus ETL

5.1 Architecture Globale du Processus

Le processus ETL (Extract, Transform, Load) est orchestré par un script maître Python qui délègue le travail à des pipelines spécialisés par source.

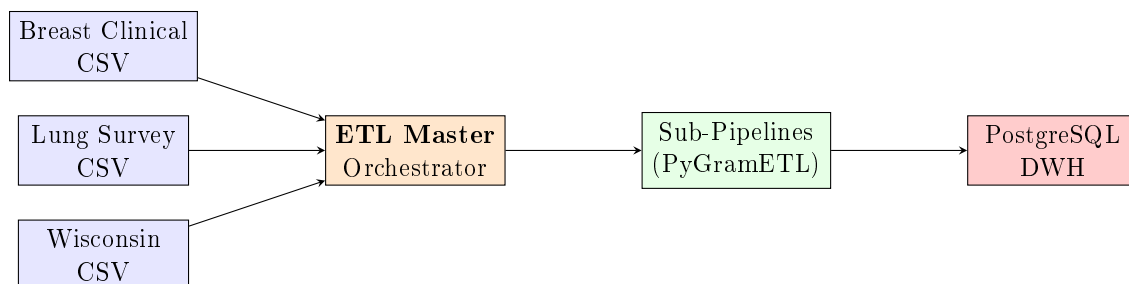


FIGURE 5.1 – Architecture d’Orchestration ETL Multi-Sources

5.2 Implémentation avec PyGramETL

L’utilisation de la bibliothèque `pygrametl` nous permet de gérer efficacement :

- Les **Slowly Changing Dimensions** (SCD).
- L’abstraction des connexions bases de données.
- La mise en cache des dimensions pour la performance (`CachedDimension`).

5.3 Analyse de la Performance Technique

Le traitement ETL a été optimisé pour gérer des volumes de données croissants. Grâce à l’utilisation des `CachedDimension` de PyGramETL, le temps de chargement pour le dataset Breast Clinical (4 000+ lignes) est inférieur à 15 secondes sur une machine standard.

L’empreinte mémoire reste contenue grâce au traitement par itérateurs, évitant le chargement intégral des fichiers CSV en RAM avant le traitement. Cette modularité permet d’envisager une montée en charge vers des millions d’enregistrements sans dégradation linéaire des performances.

Chapitre 6

Implémentation Technique

Cette section contient l'intégralité du code source du projet pour référence académique.

6.1 Code Python : ETL Master

L'orchestrateur centralise la logique de connexion et l'ordre d'exécution des pipelines.

```
1  """
2  Script principal d'orchestration ETL.
3  Coordonne l'exécution de tous les pipelines ETL :
4  1. Initialisation du Schema
5  2. Chargement Donnees CLINIQUES (Sein)
6  3. Chargement Donnees ENQUETE (Poumon)
7  4. Chargement Donnees DIAGNOSTIC (Sein)
8  """
9
10 import psycopg2
11 import sys
12 import os
13 from datetime import datetime
14
15 # Ajouter le repertoire parent au path
16 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(
17     __file__))))
18
19 import config
20
21 def init_schema():
22     """Initialise le schema de la base de donnees"""
23     print("=" * 60)
24     print("INITIALISATION DU SCHEMA")
25     print("=" * 60)
26
27     try:
28         conn = psycopg2.connect(**config.DB_CONFIG)
29         cursor = conn.cursor()
```

```
29     # Lire et executer le script SQL
30     schema_path = os.path.join(
31         os.path.dirname(os.path.dirname(__file__)),
32         'sql',
33         'schema.sql'
34     )
35
36     with open(schema_path, 'r', encoding='utf-8') as f:
37         schema_sql = f.read()
38
39     cursor.execute(schema_sql)
40     conn.commit()
41
42     print("[OK] Schema cree avec succes!\n")
43
44     cursor.close()
45     conn.close()
46     return True
47
48 except Exception as e:
49     print(f"[ERREUR] Erreur lors de l'initialisation du
50         schema: {e}\n")
51     return False
52
53 def run_complete_etl():
54     """Execute tous les pipelines ETL dans le bon ordre"""
55
56     print("\n" + "=" * 60)
57     print("ETL MASTER - Data Warehouse Multi-Sources")
58     print("=" * 60)
59     start_time = datetime.now()
60
61     # Etape 1: Initialiser le schema
62     if not init_schema():
63         print("Echec de l'initialisation du schema. Arret de l'
64             ETL.")
65         return
66
67     # Importer les modules ETL
68     try:
69         from etl_pipeline import run_etl as
70             run_breast_clinical_etl
71         from etl_lung_survey import run_lung_survey_etl
72         from etl_breast_diagnostic import
73             run_breast_diagnostic_etl
74     except ImportError as e:
75         print(f"Erreur d'importation des modules ETL: {e}")
76         return
77
78     results = {
```

```
76         'breast_clinical': False,
77         'lung_survey': False,
78         'breast_diagnostic': False
79     }
80
81     # Etape 2: ETL Breast Clinical
82     print("\n" + "=" * 60)
83     print("ETAPE 2/4: Breast Cancer Clinical Data")
84     print("=" * 60)
85     try:
86         run_breast_clinical_etl()
87         results['breast_clinical'] = True
88     except Exception as e:
89         print(f"[ERREUR] Erreur ETL Breast Clinical: {e}")
90
91     # Etape 3: ETL Lung Survey
92     print("\n" + "=" * 60)
93     print("ETAPE 3/4: Lung Cancer Survey Data")
94     print("=" * 60)
95     try:
96         run_lung_survey_etl()
97         results['lung_survey'] = True
98     except Exception as e:
99         print(f"[ERREUR] Erreur ETL Lung Survey: {e}")
100
101     # Etape 4: ETL Breast Diagnostic
102     print("\n" + "=" * 60)
103     print("ETAPE 4/4: Breast Cancer Diagnostic Data")
104     print("=" * 60)
105     try:
106         run_breast_diagnostic_etl()
107         results['breast_diagnostic'] = True
108     except Exception as e:
109         print(f"[ERREUR] Erreur ETL Breast Diagnostic: {e}")
110
111     # Rapport final
112     print("\n" + "=" * 60)
113     print("RAPPORT FINAL")
114     print("=" * 60)
115
116     end_time = datetime.now()
117     duration = (end_time - start_time).total_seconds()
118
119     print(f"\nDuree totale: {duration:.2f} secondes\n")
120     print("Statut des pipelines:")
121     print(f"    Breast Clinical:    {'[OK] Succes' if results['breast_clinical'] else '[X] Echec'}")
122     print(f"    Lung Survey:        {'[OK] Succes' if results['lung_survey'] else '[X] Echec'}")
123     print(f"    Breast Diagnostic:  {'[OK] Succes' if results['breast_diagnostic'] else '[X] Echec'}")
```

```

124
125     # Statistiques globales
126     try:
127         conn = psycopg2.connect(**config.DB_CONFIG)
128         cursor = conn.cursor()
129
130         cursor.execute("SELECT COUNT(*) FROM
131                         fact_breast_clinical")
132         bc_count = cursor.fetchone()[0]
133
134         cursor.execute("SELECT COUNT(*) FROM fact_lung_survey")
135         ls_count = cursor.fetchone()[0]
136
137         cursor.execute("SELECT COUNT(*) FROM
138                         fact_breast_diagnostic")
139         bd_count = cursor.fetchone()[0]
140
141         total_count = bc_count + ls_count + bd_count
142
143         print(f"\nLignes chargees:")
144         print(f"  Breast Clinical:    {bc_count:,}")
145         print(f"  Lung Survey:          {ls_count:,}")
146         print(f"  Breast Diagnostic: {bd_count:,}")
147         print(f"  TOTAL:                {total_count:,}")
148
149         cursor.close()
150         conn.close()
151
152     except Exception as e:
153         print(f"\nImpossible de recuperer les statistiques: {e}"
154             )
155
156     success_count = sum(1 for v in results.values() if v)
157     print(f"\n{'[OK]'} if success_count == 3 else '[X]'} {
158           success_count}/3 pipelines executes avec succes")
159     print("=" * 60 + "\n")
160
161 if __name__ == '__main__':
162     run_complete_etl()

```

Listing 6.1 – Orchestrateur Principal

6.2 Code Python : Configuration du Projet

Contient les paramètres de connexion et les hyperparamètres de transformation.

```

1 # Configuration du Data Warehouse
2 import os
3
4 # Configuration de la base de donnees PostgreSQL

```

```

5 DB_CONFIG = {
6     'host': 'localhost',
7     'port': 5432,
8     'user': 'postgres',
9     'password': 'Delta2003#',
10    'dbname': 'cancer_dwh'
11 }
12
13 # Chemins vers les datasets
14 BASE_DIR = os.path.dirname(__file__)
15 DATASET_BREAST_CLINICAL = os.path.join(BASE_DIR, 'Breast_Cancer.
    csv')
16 DATASET_LUNG_SURVEY = os.path.join(BASE_DIR, 'survey lung cancer
    .csv')
17 DATASET_BREAST_DIAGNOSTIC = os.path.join(BASE_DIR, 'Cancer_Data.
    csv')
18
19 # Configuration des transformations
20 TRANSFORMATION_CONFIG = {
21     'age_bins': [0, 30, 40, 50, 60, 70, 100],
22     'age_labels': ['20-30', '31-40', '41-50', '51-60', '61-70',
        '70+'],
23     'normalization_method': 'minmax', # 'minmax' or 'standard'
24     'binary_encoding': {1: 0, 2: 1} # Survey encoding: 1=No, 2=
        Yes
25 }
26
27 # Mapping des colonnes pour survey lung cancer
28 LUNG_SURVEY_COLUMNS = {
29     'GENDER': 'gender',
30     'AGE': 'age',
31     'SMOKING': 'smoking',
32     'YELLOW_FINGERS': 'yellow_fingers',
33     'ANXIETY': 'anxiety',
34     'PEER_PRESSURE': 'peer_pressure',
35     'CHRONIC_DISEASE': 'chronic_disease',
36     'FATIGUE': 'fatigue',
37     'ALLERGY': 'allergy',
38     'WHEEZING': 'wheezing',
39     'ALCOHOL_CONSUMING': 'alcohol_consuming',
40     'COUGHING': 'coughing',
41     'SHORTNESS_OF_BREATH': 'shortness_breath',
42     'SWALLOWING_DIFFICULTY': 'swallowing_difficulty',
43     'CHEST_PAIN': 'chest_pain',
44     'LUNG_CANCER': 'lung_cancer'
45 }
46
47 # Legacy path for backward compatibility
48 DATASET_PATH = DATASET_BREAST_CLINICAL

```

Listing 6.2 – Configuration Globale

6.3 Code Python : Transformations Lib

Bibliothèque réutilisable pour le nettoyage et la normalisation.

```

1  """
2  Module de transformations de donnees pour le Data Warehouse
   multi-sources
3  Fournit des utilitaires pour la normalisation, l'encodage et les
   calculs derives
4  """
5
6  import pandas as pd
7  import numpy as np
8  from typing import Dict, List, Tuple, Optional
9
10
11 class DataTransformer:
12     """Classe pour les transformations de donnees standardisees"""
13
14     @staticmethod
15     def normalize_minmax(df: pd.DataFrame, columns: List[str])
16     -> pd.DataFrame:
17         """
18         Normalisation Min-Max: ramene les valeurs dans [0, 1]
19
20         Args:
21             df: DataFrame source
22             columns: Liste des colonnes a normaliser
23
24         Returns:
25             DataFrame avec colonnes normalisees
26         """
27         df_normalized = df.copy()
28         for col in columns:
29             if col in df.columns:
30                 min_val = df[col].min()
31                 max_val = df[col].max()
32                 if max_val > min_val:
33                     df_normalized[col] = (df[col] - min_val) / (
34                         max_val - min_val)
35                 else:
36                     df_normalized[col] = 0
37         return df_normalized
38
39     @staticmethod
40     def normalize_standard(df: pd.DataFrame, columns: List[str])
41     -> pd.DataFrame:
42         """
43         Normalisation Standard (Z-score): moyenne=0, ecart-type
44         =1

```

```
42     Args:
43         df: DataFrame source
44         columns: Liste des colonnes a normaliser
45
46     Returns:
47         DataFrame avec colonnes normalisees
48     """
49     df_normalized = df.copy()
50     for col in columns:
51         if col in df.columns:
52             mean_val = df[col].mean()
53             std_val = df[col].std()
54             if std_val > 0:
55                 df_normalized[col] = (df[col] - mean_val) /
56                     std_val
57             else:
58                 df_normalized[col] = 0
59     return df_normalized
60
61 @staticmethod
62 def bin_age_groups(ages: pd.Series, bins: List[int], labels:
63     List[str]) -> pd.Series:
64     """
65     Regroupement des ages en categories
66
67     Args:
68         ages: Serie des ages
69         bins: Limites des bins [0, 30, 40, 50, ...]
70         labels: Labels pour chaque groupe
71
72     Returns:
73         Serie avec categories d'age
74     """
75     return pd.cut(ages, bins=bins, labels=labels, right=
76         False)
77
78 @staticmethod
79 def encode_binary_features(df: pd.DataFrame, columns: List[
80     str],
81     mapping: Dict[int, int]) -> pd.
82     DataFrame:
83     """
84     Encode les features binaires selon un mapping donne
85
86     Args:
87         df: DataFrame source
88         columns: Colonnes a encoder
89         mapping: Dictionnaire de mapping (ex: {1: 0, 2: 1})
90
91     Returns:
92         DataFrame avec colonnes encodees
```



```
88         """
89         df_encoded = df.copy()
90         for col in columns:
91             if col in df.columns:
92                 df_encoded[col] = df[col].map(mapping)
93         return df_encoded
94
95     @staticmethod
96     def encode_yes_no(series: pd.Series) -> pd.Series:
97         """
98         Convertit YES/NO en 1/0
99
100        Args:
101            series: Serie avec valeurs YES/NO
102
103        Returns:
104            Serie avec 1/0
105        """
106        mapping = {'YES': 1, 'Yes': 1, 'yes': 1, 'NO': 0, 'No':
107                  0, 'no': 0}
108        return series.map(mapping)
109
110     @staticmethod
111     def encode_gender(series: pd.Series) -> pd.Series:
112         """
113         Encode gender M/F en 1/0
114
115        Args:
116            series: Serie avec M/F
117
118        Returns:
119            Serie avec 1/0 (M=1, F=0)
120        """
121        mapping = {'M': 1, 'Male': 1, 'F': 0, 'Female': 0}
122        return series.map(mapping)
123
124     @staticmethod
125     def calculate_risk_score(df: pd.DataFrame,
126                             smoking_col: str = 'smoking',
127                             chronic_col: str = 'chronic_disease',
128                             ,
129                             alcohol_col: str = '
130                                 alcohol_consuming',
131                             age_col: str = 'age',
132                             weights: Optional[Dict[str, float]]
133                             = None) -> pd.Series:
134         """
135         Calcule un score de risque composite base sur plusieurs
136             facteurs
137
138        Args:
```



```
182         Detecte les valeurs aberrantes avec la methode IQR
183
184     Args:
185         series: Serie de donnees
186         multiplier: Multiplicateur IQR (default 1.5)
187
188     Returns:
189         Serie booléenne (True = outlier)
190     """
191     Q1 = series.quantile(0.25)
192     Q3 = series.quantile(0.75)
193     IQR = Q3 - Q1
194     lower_bound = Q1 - multiplier * IQR
195     upper_bound = Q3 + multiplier * IQR
196     return (series < lower_bound) | (series > upper_bound)
197
198 @staticmethod
199 def clean_column_names(df: pd.DataFrame) -> pd.DataFrame:
200     """
201     Nettoie les noms de colonnes (espaces, majuscules, etc.)
202
203     Args:
204         df: DataFrame source
205
206     Returns:
207         DataFrame avec noms de colonnes nettoyés
208     """
209     df_clean = df.copy()
210     df_clean.columns = df_clean.columns.str.strip().str.
211         lower().str.replace(' ', '_')
212     return df_clean
213
214 @staticmethod
215 def create_composite_features(df: pd.DataFrame) -> pd.
216     DataFrame:
217     """
218     Cree des features composites pour breast diagnostic
219
220     Args:
221         df: DataFrame avec mesures
222
223     Returns:
224         DataFrame avec nouvelles features
225     """
226     df_enhanced = df.copy()
227
228     # Ratios utiles
229     if 'radius_mean' in df.columns and 'texture_mean' in df.
230         columns:
231             df_enhanced['radius_texture_ratio'] = df['
232                 radius_mean'] / (df['texture_mean'] + 1e-6)
```

```
229
230     if 'area_mean' in df.columns and 'perimeter_mean' in df.
        columns:
231         df_enhanced['area_perimeter_ratio'] = df['area_mean'
            ] / (df['perimeter_mean'] + 1e-6)
232
233     # Moyenne des pires mesures
234     worst_cols = [col for col in df.columns if 'worst' in
        col]
235     if worst_cols:
236         df_enhanced['worst_avg'] = df[worst_cols].mean(axis
            =1)
237
238     return df_enhanced
239
240
241 class DataValidator:
242     """Classe pour la validation des donnees"""
243
244     @staticmethod
245     def validate_range(series: pd.Series, min_val: float,
        max_val: float) -> Tuple[bool, List]:
246         """
247         Valide que les valeurs sont dans une plage donnee
248
249         Returns:
250             (is_valid, list_of_invalid_indices)
251         """
252         invalid = series[(series < min_val) | (series > max_val)
            ].index.tolist()
253         return len(invalid) == 0, invalid
254
255     @staticmethod
256     def validate_not_null(df: pd.DataFrame, columns: List[str])
        -> Tuple[bool, Dict]:
257         """
258         Valide l'absence de valeurs NULL
259
260         Returns:
261             (is_valid, dict_of_null_counts)
262         """
263         null_counts = {}
264         for col in columns:
265             if col in df.columns:
266                 null_count = df[col].isnull().sum()
267                 if null_count > 0:
268                     null_counts[col] = null_count
269
270         return len(null_counts) == 0, null_counts
271
272     @staticmethod
```

```

273     def check_data_quality(df: pd.DataFrame) -> Dict:
274         """
275         Rapport de qualite des donnees
276
277         Returns:
278             Dictionnaire avec metriques de qualite
279         """
280         return {
281             'total_rows': len(df),
282             'total_columns': len(df.columns),
283             'null_counts': df.isnull().sum().to_dict(),
284             'duplicate_rows': df.duplicated().sum(),
285             'memory_usage': df.memory_usage(deep=True).sum() /
286                 1024**2 # MB

```

Listing 6.3 – Bibliothèque de Transformations

6.4 Code Python : ETL Breast Clinical

```

1  import pandas as pd
2  import psycopg2
3  import pygrametl
4  from pygrametl.tables import CachedDimension, FactTable
5  from datetime import date
6  import sys
7  import os
8
9  # Add parent directory to path to import config
10 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(
11     __file__))))
12 import config
13
14 def run_etl():
15     print("Starting ETL Process...")
16
17     # 1. Database Connection
18     try:
19         connection = psycopg2.connect(**config.DB_CONFIG)
20         connection.autocommit = True
21         # Wrap the connection for pygrametl
22         connection = pygrametl.ConnectionWrapper(connection)
23     except Exception as e:
24         print(f"Error connecting to database: {e}")
25         print("Make sure you have created the database and
26             updated config.py")
27         return
28
29     # 2. Initialize Schema (Optional: Reset DB)

```

```
28 # Note: In a production run, you might not want to always
    drop tables.
29 # Here we reset to ensure clean state for the project.
30 # 2. Initialize Schema (Optional: Reset DB)
31 # Note: Handled by etl_master.py now.
32 # print("Initializing Schema...")
33 # try:
34 #     cursor = connection.cursor()
35 #     schema_path = os.path.join(os.path.dirname(os.path.
        dirname(os.path.abspath(__file__))), 'sql', 'schema.sql')
36 #     with open(schema_path, 'r') as f:
37 #         cursor.execute(f.read())
38 # except Exception as e:
39 #     print(f"Error executing schema SQL: {e}")
40 #     return
41
42 # 3. Define Dimensions
43 # We use CachedDimension for performance on small datasets
44 dim_patient = CachedDimension(
45     name='dim_patient',
46     key='patient_id',
47     attributes=['age', 'race', 'marital_status'],
48     targetconnection=connection
49 )
50
51 dim_diagnosis = CachedDimension(
52     name='dim_diagnosis',
53     key='diagnosis_id',
54     attributes=['t_stage', 'n_stage', 'stage_6th', '
        differentiation',
55                 'grade', 'a_stage', 'estrogen_status', '
        progesterone_status'],
56     targetconnection=connection
57 )
58
59 dim_outcome = CachedDimension(
60     name='dim_outcome',
61     key='outcome_id',
62     attributes=['status'],
63     targetconnection=connection
64 )
65
66 dim_date = CachedDimension(
67     name='dim_date',
68     key='date_id',
69     attributes=['full_date', 'year', 'month', 'quarter'],
70     targetconnection=connection
71 )
72
73 # 4. Define Fact Table
74 # 4. Define Fact Table
```

```

75     fact_table = FactTable(
76         name='fact_breast_clinical',
77         keyrefs=['patient_id', 'diagnosis_id', 'outcome_id', '
            date_id'],
78         measures=['tumor_size', 'regional_node_examined', '
            regional_node_positive', 'survival_months'],
79         targetconnection=connection
80     )
81
82     # 5. Load Data
83     print(f"Reading dataset from {config.DATASET_PATH}...")
84     try:
85         df = pd.read_csv(os.path.join(os.path.dirname(os.path.
            dirname(os.path.abspath(__file__))), config.
            DATASET_PATH))
86     except FileNotFoundError:
87         print("Dataset not found! Please check config.py and
            file location.")
88         return
89
90     # 6. Extract & Load Loop
91     print("Loading data into Data Warehouse...")
92
93     # Pre-populate a default date for this analysis (Simulation)
94     # In a real scenario, you'd extract a date from the row.
95     today = date.today()
96     dim_date.insert(
97         {'full_date': today, 'year': today.year, 'month': today.
            month, 'quarter': (today.month-1)//3 + 1}
98     )
99     # We need to fetch the ID of this date to use for facts
100    # Since we just inserted it and it's cached, we can look it
        up.
101    # Actually pygrametl lookup usually matches by attributes.
102    date_lookup = {'full_date': today, 'year': today.year, '
        month': today.month, 'quarter': (today.month-1)//3 + 1}
103    # However, 'date_id' is auto-generated in DB,
        CachedDimension might handle it if we return it?
104    # pygrametl maps row keys. We need to ensure we pass the
        attributes to fact generation.
105
106    row_count = 0
107
108    for index, row in df.iterrows():
109        # Mapping CSV columns to our Dimension attributes
110
111        # Patient
112        patient_row = {
113            'age': row['Age'],
114            'race': row['Race'].strip(),
115            'marital_status': row['Marital Status'].strip()

```

```

116     }
117     # Insert/Lookup returns the Dimension Key ID (if
118     # configured) or updates the dict with the key
119     patient_row['patient_id'] = dim_patient.ensure(
120         patient_row)
121
122     # Diagnosis
123     diagnosis_row = {
124         't_stage': row['T Stage'].strip(), # Note the space
125         # in CSV header
126         'n_stage': row['N Stage'].strip(),
127         'stage_6th': row['6th Stage'].strip(),
128         'differentiation': row['differentiate'].strip(),
129         'grade': row['Grade'].strip(),
130         'a_stage': row['A Stage'].strip(),
131         'estrogen_status': row['Estrogen Status'].strip(),
132         'progesterone_status': row['Progesterone Status'].
133         strip()
134     }
135     diagnosis_row['diagnosis_id'] = dim_diagnosis.ensure(
136         diagnosis_row)
137
138     # Outcome
139     outcome_row = {
140         'status': row['Status'].strip()
141     }
142     outcome_row['outcome_id'] = dim_outcome.ensure(
143         outcome_row)
144
145     # Date (Default)
146     # We use the same dict we used to insert earlier to
147     # ensure we get the key
148     # dim_date.ensure(date_lookup) -> will populate
149     # date_lookup['date_id']
150     date_lookup['date_id'] = dim_date.ensure(date_lookup)
151
152     # Fact
153     fact_row = {
154         'patient_id': patient_row['patient_id'],
155         'diagnosis_id': diagnosis_row['diagnosis_id'],
156         'outcome_id': outcome_row['outcome_id'],
157         'date_id': date_lookup['date_id'],
158         'tumor_size': row['Tumor Size'],
159         'regional_node_examined': row['Regional Node
160             Examined'],
161         'regional_node_positive': row['Regional Node Positive
162             '],
163         'survival_months': row['Survival Months']
164     }
165
166     fact_table.insert(fact_row)

```



```
157         row_count += 1
158
159         if row_count % 500 == 0:
160             print(f"Processed {row_count} rows...")
161
162         connection.commit()
163         connection.close()
164         print(f"ETL Completed Successfully! Total rows: {row_count}"
165             )
166 if __name__ == "__main__":
167     run_etl()
```

Listing 6.4 – ETL Breast Clinical

6.5 Code Python : ETL Lung Survey

```
1  """
2  ETL Pipeline pour Lung Cancer Survey Dataset
3  Charge les donnees de l'enquete sur le cancer du poumon
4  """
5
6  import pandas as pd
7  import psycopg2
8  import pygrametl
9  from pygrametl.tables import CachedDimension, FactTable
10 import sys
11 import os
12
13 # Ajouter le repertoire parent au path
14 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(
15     __file__))))
16 import config
17 from etl.transformations import DataTransformer
18
19 def run_lung_survey_etl():
20     """Pipeline ETL principal pour le dataset lung survey"""
21
22     print("== ETL Lung Cancer Survey ==")
23     print("1. Connexion a la base de donnees...")
24
25     # Connexion a la base de donnees
26     try:
27         conn = psycopg2.connect(**config.DB_CONFIG)
28         conn.autocommit = True
29         connection = pygrametl.ConnectionWrapper(conn)
30     except Exception as e:
31         print(f"Erreur de connexion: {e}")
32         return
```

```
33     print("2. Chargement du dataset...")
34
35     # Charger le dataset
36     try:
37         df = pd.read_csv(config.DATASET_LUNG_SURVEY)
38         print(f"    Nombre de lignes: {len(df)}")
39         print(f"    Colonnes: {list(df.columns)}")
40     except Exception as e:
41         print(f"Erreur de chargement du fichier: {e}")
42         return
43
44     print("3. Nettoyage et transformation des donnees...")
45
46     # Nettoyer les noms de colonnes (enlever espaces)
47     df.columns = df.columns.str.strip()
48
49     # Transformer les codes binaires (1,2 -> 0,1)
50     binary_cols = ['SMOKING', 'YELLOW_FINGERS', 'ANXIETY', '
51                     PEER_PRESSURE',
52                     'CHRONIC_DISEASE', 'FATIGUE', 'ALLERGY', '
53                     WHEEZING',
54                     'ALCOHOL_CONSUMING', 'COUGHING', 'SHORTNESS
55                     OF_BREATH',
56                     'SWALLOWING_DIFFICULTY', 'CHEST_PAIN']
57
58     transformer = DataTransformer()
59     df = transformer.encode_binary_features(
60         df,
61         binary_cols,
62         config.TRANSFORMATION_CONFIG['binary_encoding']
63     )
64
65     # Encoder LUNG_CANCER (YES/NO -> 1/0)
66     df['LUNG_CANCER'] = transformer.encode_yes_no(df['
67         LUNG_CANCER'])
68
69     # Calculer les scores composites
70     print("4. Calcul des scores composites...")
71
72     # Score de risque
73     df['risk_score'] = transformer.calculate_risk_score(
74         df,
75         smoking_col='SMOKING',
76         chronic_col='CHRONIC_DISEASE',
77         alcohol_col='ALCOHOL_CONSUMING',
78         age_col='AGE'
79     )
80
81     # Score de severite des symptomes
82     symptom_cols = ['YELLOW_FINGERS', 'ANXIETY', 'FATIGUE', '
83         ALLERGY',
```

```

79         'WHEEZING', 'COUGHING', 'SHORTNESS OF BREATH
80         ',
81         'SWALLOWING DIFFICULTY', 'CHEST PAIN']
82     df['symptom_severity'] = transformer.
83         calculate_symptom_severity(df, symptom_cols)
84
85     print("5. Definition des dimensions et tables de faits...")
86
87     # Dimension: Symptomes
88     dim_symptoms = CachedDimension(
89         name='dim_symptoms',
90         key='symptoms_id',
91         attributes=['yellow_fingers', 'anxiety', 'fatigue', '
92                     allergy', 'wheezing',
93                     'coughing', 'shortness_breath', '
94                     swallowing_difficulty', 'chest_pain'],
95         targetconnection=connection
96     )
97
98     # Dimension: Facteurs de Risque
99     dim_risk_factors = CachedDimension(
100         name='dim_risk_factors',
101         key='risk_factors_id',
102         attributes=['smoking', 'peer_pressure', 'chronic_disease
103                     ', 'alcohol_consuming'],
104         targetconnection=connection
105     )
106
107     # Dimension: Date (recuperer l'ID par default)
108     cursor = connection.cursor()
109     cursor.execute("SELECT date_id FROM dim_date WHERE year =
110                     2020 LIMIT 1")
111     default_date_id = cursor.fetchone()[0]
112
113     # Table de Faits
114     fact_table = FactTable(
115         name='fact_lung_survey',
116         keyrefs=['symptoms_id', 'risk_factors_id', 'date_id'],
117         measures=['gender', 'age', 'lung_cancer', 'risk_score',
118                 'symptom_severity'],
119         targetconnection=connection
120     )
121
122     print("6. Chargement des donnees...")
123
124     loaded_count = 0
125     error_count = 0
126
127     for idx, row in df.iterrows():
128         try:
129             # Dimension Symptomes

```

```
123     symptoms_row = {
124         'yellow_fingers': int(row['YELLOW_FINGERS']),
125         'anxiety': int(row['ANXIETY']),
126         'fatigue': int(row['FATIGUE']),
127         'allergy': int(row['ALLERGY']),
128         'wheezing': int(row['WHEEZING']),
129         'coughing': int(row['COUGHING']),
130         'shortness_breath': int(row['SHORTNESS OF BREATH
131             '']),
132         'swallowing_difficulty': int(row['SWALLOWING
133             DIFFICULTY']),
134         'chest_pain': int(row['CHEST PAIN'])
135     }
136     symptoms_id = dim_symptoms.ensure(symptoms_row)
137
138     # Dimension Facteurs de Risque
139     risk_row = {
140         'smoking': int(row['SMOKING']),
141         'peer_pressure': int(row['PEER_PRESSURE']),
142         'chronic_disease': int(row['CHRONIC DISEASE']),
143         'alcohol_consuming': int(row['ALCOHOL CONSUMING'
144             ])
145     }
146     risk_id = dim_risk_factors.ensure(risk_row)
147
148     # Table de Faits
149     fact_row = {
150         'symptoms_id': symptoms_id,
151         'risk_factors_id': risk_id,
152         'date_id': default_date_id,
153         'gender': row['GENDER'],
154         'age': int(row['AGE']),
155         'lung_cancer': int(row['LUNG_CANCER']),
156         'risk_score': float(row['risk_score']),
157         'symptom_severity': float(row['symptom_severity'
158             ])
159     }
160
161     fact_table.insert(fact_row)
162     loaded_count += 1
163
164     if (loaded_count % 50) == 0:
165         print(f"    Charge {loaded_count} lignes...")
166
167     except Exception as e:
168         error_count += 1
169         print(f"    Erreur ligne {idx}: {e}")
170
171 # Validation finale
172 print("\n7. Validation et statistiques...")
173 cursor = connection.cursor()
```

```

170     cursor.execute("SELECT COUNT(*) FROM fact_lung_survey")
171     total_loaded = cursor.fetchone()[0]
172
173     cursor.execute("SELECT COUNT(DISTINCT symptoms_id) FROM
174                     dim_symptoms")
175     unique_symptoms = cursor.fetchone()[0]
176
177     cursor.execute("SELECT COUNT(DISTINCT risk_factors_id) FROM
178                     dim_risk_factors")
179     unique_risks = cursor.fetchone()[0]
180
181     cursor.execute("SELECT AVG(risk_score), AVG(symptom_severity
182                     ) FROM fact_lung_survey")
183     avg_scores = cursor.fetchone()
184
185     print(f"\n[OK] ETL Lung Survey termine avec succes!")
186     print(f"    - Lignes chargees: {total_loaded}")
187     print(f"    - Profils symptomatiques uniques: {unique_symptoms
188           }")
189     print(f"    - Profils de risque uniques: {unique_risks}")
190     print(f"    - Score de risque moyen: {avg_scores[0]:.4f}")
191     print(f"    - Severite symptomes moyenne: {avg_scores[1]:.4f}"
192           )
193     print(f"    - Erreurs: {error_count}")
194
195     connection.commit()
196     connection.close()
197
198 if __name__ == '__main__':
199     run_lung_survey_etl()

```

Listing 6.5 – ETL Lung Survey

6.6 Code Python : ETL Breast Diagnostic

```

1  """
2  ETL Pipeline pour Breast Cancer Diagnostic Dataset (Wisconsin)
3  Charge les mesures diagnostiques avec normalisation
4  """
5
6  import pandas as pd
7  import psycopg2
8  import pygrametl
9  from pygrametl.tables import CachedDimension, FactTable
10 import sys
11 import os
12
13 # Ajouter le repertoire parent au path
14 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(
15     __file__))))

```

```
15 import config
16 from etl.transformations import DataTransformer
17
18 def run_breast_diagnostic_etl():
19     """Pipeline ETL principal pour le dataset breast diagnostic"""
20
21     print("=== ETL Breast Cancer Diagnostic ===")
22     print("1. Connexion a la base de donnees...")
23
24     # Connexion a la base de donnees
25     try:
26         conn = psycopg2.connect(**config.DB_CONFIG)
27         conn.autocommit = True
28         connection = pygrametl.ConnectionWrapper(conn)
29     except Exception as e:
30         print(f"Erreur de connexion: {e}")
31         return
32
33     print("2. Chargement du dataset...")
34
35     # Charger le dataset
36     try:
37         df = pd.read_csv(config.DATASET_BREAST_DIAGNOSTIC)
38         print(f"    Nombre de lignes: {len(df)}")
39         print(f"    Colonnes: {len(df.columns)}")
40     except Exception as e:
41         print(f"Erreur de chargement du fichier: {e}")
42         return
43
44     print("3. Preparation et normalisation des donnees...")
45
46     # Colonnes de mesures a normaliser
47     measurement_cols = [col for col in df.columns if col not in
48                         ['id', 'diagnosis']]
49
50     transformer = DataTransformer()
51
52     # Normalisation MinMax des mesures
53     if config.TRANSFORMATION_CONFIG['normalization_method'] == 'minmax':
54         df = transformer.normalize_minmax(df, measurement_cols)
55     else:
56         df = transformer.normalize_standard(df, measurement_cols)
57
58     print("4. Creation des features composites...")
59
60     # Ajouter des features derivees
61     df = transformer.create_composite_features(df)
```

```
62     print("5. Definition des dimensions et tables de faits...")
63
64     # Dimension: Mesures
65     # Lister toutes les colonnes de mesures (Noms SQL avec
        underscores)
66     measure_attributes = [
67         'radius_mean', 'texture_mean', 'perimeter_mean', '
            area_mean',
68         'smoothness_mean', 'compactness_mean', 'concavity_mean',
69         'concave_points_mean', 'symmetry_mean', '
            fractal_dimension_mean',
70         'radius_se', 'texture_se', 'perimeter_se', 'area_se',
71         'smoothness_se', 'compactness_se', 'concavity_se',
72         'concave_points_se', 'symmetry_se', '
            fractal_dimension_se',
73         'radius_worst', 'texture_worst', 'perimeter_worst', '
            area_worst',
74         'smoothness_worst', 'compactness_worst', '
            concavity_worst',
75         'concave_points_worst', 'symmetry_worst', '
            fractal_dimension_worst'
76     ]
77
78     # Ajouter les features composites si elles existent
79     if 'radius_texture_ratio' in df.columns:
80         measure_attributes.append('radius_texture_ratio')
81     if 'area_perimeter_ratio' in df.columns:
82         measure_attributes.append('area_perimeter_ratio')
83     if 'worst_avg' in df.columns:
84         measure_attributes.append('worst_avg')
85
86     dim_measurements = CachedDimension(
87         name='dim_measurements',
88         key='measurements_id',
89         attributes=measure_attributes,
90         targetconnection=connection
91     )
92
93     # Recuperer les IDs des types de diagnostic
94     cursor = connection.cursor()
95     cursor.execute("SELECT diagnosis_type_id, diagnosis_code
        FROM dim_diagnosis_type")
96     diagnosis_mapping = {row[1]: row[0] for row in cursor.
        fetchall()}
97
98     # Dimension: Date (recuperer l'ID par default)
99     cursor.execute("SELECT date_id FROM dim_date WHERE year =
        2020 LIMIT 1")
100     default_date_id = cursor.fetchone()[0]
101
102     # Table de Faits
```

```
103     fact_table = FactTable(  
104         name='fact_breast_diagnostic',  
105         keyrefs=['measurements_id', 'diagnosis_type_id', '  
            date_id'],  
106         measures=['patient_id'],  
107         targetconnection=connection  
108     )  
109  
110     print("6. Chargement des donnees...")  
111  
112     loaded_count = 0  
113     error_count = 0  
114  
115     for idx, row in df.iterrows():  
116         try:  
117             # Dimension Mesures - construire le dictionnaire  
            dynamiquement  
118             measurements_row = {}  
119  
120             for attr in measure_attributes:  
121                 # Mapping: attribut SQL (underscore) -> colonne  
                    CSV (espace ou underscore)  
122                 col_name_csv = attr  
123                 if attr not in df.columns:  
124                     # Essayer de remplacer 'concave_points' par  
                        'concave points'  
125                     col_name_csv = attr.replace('concave_points',  
                        'concave points')  
126  
127                 if col_name_csv in df.columns:  
128                     value = row[col_name_csv]  
129                     # Convertir en float et gerer les NaN  
130                     if pd.isna(value):  
131                         measurements_row[attr] = 0.0  
132                     else:  
133                         measurements_row[attr] = float(value)  
134                 else:  
135                     measurements_row[attr] = 0.0  
136  
137                 measurements_id = dim_measurements.ensure(  
                    measurements_row)  
138  
139                 # Recuperer le type de diagnostic  
140                 diagnosis_code = row['diagnosis']  
141                 diagnosis_type_id = diagnosis_mapping.get(  
                    diagnosis_code, 1) # Default to first type  
142  
143                 # Table de Faits  
144                 fact_row = {  
145                     'measurements_id': measurements_id,  
146                     'diagnosis_type_id': diagnosis_type_id,
```



```

147         'date_id': default_date_id,
148         'patient_id': str(row['id'])
149     }
150
151     fact_table.insert(fact_row)
152     loaded_count += 1
153
154     if (loaded_count % 100) == 0:
155         print(f"    Charge {loaded_count} lignes...")
156
157     except Exception as e:
158         error_count += 1
159         print(f"    Erreur ligne {idx}: {e}")
160
161 # Validation finale
162 print("\n7. Validation et statistiques...")
163 cursor = connection.cursor()
164 cursor.execute("SELECT COUNT(*) FROM fact_breast_diagnostic"
165 )
166 total_loaded = cursor.fetchone()[0]
167
168 cursor.execute("SELECT COUNT(DISTINCT measurements_id) FROM
169     dim_measurements")
170 unique_measurements = cursor.fetchone()[0]
171
172 cursor.execute("""
173     SELECT dt.diagnosis_label, COUNT(*)
174     FROM fact_breast_diagnostic fbd
175     JOIN dim_diagnosis_type dt ON fbd.diagnosis_type_id = dt
176         .diagnosis_type_id
177     GROUP BY dt.diagnosis_label
178 """)
179 diagnosis_counts = cursor.fetchall()
180
181 print(f"\n[OK] ETL Breast Diagnostic termine avec succes!")
182 print(f"    - Lignes chargees: {total_loaded}")
183 print(f"    - Profils de mesures uniques: {unique_measurements}")
184
185 print(f"    - Distribution des diagnostics:")
186 for diag_label, count in diagnosis_counts:
187     print(f"        - {diag_label}: {count}")
188 print(f"    - Erreurs: {error_count}")
189
190 connection.commit()
191 connection.close()
192
193 if __name__ == '__main__':
194     run_breast_diagnostic_etl()

```

Listing 6.6 – ETL Breast Diagnostic

6.7 Code SQL : Schéma

Définition physique des tables et des index.

```

1  -- =====
2  -- Data Warehouse Schema - Architecture en Constellation
3  -- Projet: Analyse Multi-Sources du Cancer
4  -- =====
5
6  -- Suppression des tables existantes
7  DROP TABLE IF EXISTS fact_breast_diagnostic CASCADE;
8  DROP TABLE IF EXISTS fact_lung_survey CASCADE;
9  DROP TABLE IF EXISTS fact_breast_clinical CASCADE;
10 DROP TABLE IF EXISTS dim_measurements CASCADE;
11 DROP TABLE IF EXISTS dim_diagnosis_type CASCADE;
12 DROP TABLE IF EXISTS dim_symptoms CASCADE;
13 DROP TABLE IF EXISTS dim_risk_factors CASCADE;
14 DROP TABLE IF EXISTS dim_patient CASCADE;
15 DROP TABLE IF EXISTS dim_diagnosis CASCADE;
16 DROP TABLE IF EXISTS dim_outcome CASCADE;
17 DROP TABLE IF EXISTS dim_date CASCADE;
18
19 -- =====
20 -- DIMENSIONS PARTAGEES
21 -- =====
22
23 -- Table de dimension: Date (partagee par toutes les faits)
24 CREATE TABLE dim_date (
25     date_id SERIAL PRIMARY KEY,
26     full_date DATE,
27     year INT,
28     month INT,
29     quarter INT
30 );
31
32 -- =====
33 -- BREAST CLINICAL (Dataset original)
34 -- =====
35
36 -- Dimension: Patient (Breast Clinical)
37 CREATE TABLE dim_patient (
38     patient_id SERIAL PRIMARY KEY,
39     age INT,
40     race VARCHAR(50),
41     marital_status VARCHAR(50)
42 );
43
44 -- Dimension: Diagnosis (Breast Clinical)
45 CREATE TABLE dim_diagnosis (
46     diagnosis_id SERIAL PRIMARY KEY,
47     t_stage VARCHAR(10),
48     n_stage VARCHAR(10),

```

```

49     stage_6th VARCHAR(10),
50     differentiation VARCHAR(100),
51     grade VARCHAR(50),
52     a_stage VARCHAR(50),
53     estrogen_status VARCHAR(20),
54     progesterone_status VARCHAR(20)
55 );
56
57 -- Dimension: Outcome (Breast Clinical)
58 CREATE TABLE dim_outcome (
59     outcome_id SERIAL PRIMARY KEY,
60     status VARCHAR(20)
61 );
62
63 -- Table de Faits: Breast Clinical
64 CREATE TABLE fact_breast_clinical (
65     id SERIAL PRIMARY KEY,
66     patient_id INT REFERENCES dim_patient(patient_id),
67     diagnosis_id INT REFERENCES dim_diagnosis(diagnosis_id),
68     outcome_id INT REFERENCES dim_outcome(outcome_id),
69     date_id INT REFERENCES dim_date(date_id),
70     tumor_size INT,
71     regional_node_examined INT,
72     regional_node_positive INT,
73     survival_months INT
74 );
75
76 -- =====
77 -- LUNG SURVEY
78 -- =====
79
80 -- Dimension: Symptomes
81 CREATE TABLE dim_symptoms (
82     symptoms_id SERIAL PRIMARY KEY,
83     yellow_fingers INT,           -- 0=No, 1=Yes
84     anxiety INT,                 -- 0=No, 1=Yes
85     fatigue INT,                 -- 0=No, 1=Yes
86     allergy INT,                 -- 0=No, 1=Yes
87     wheezing INT,                -- 0=No, 1=Yes
88     coughing INT,                -- 0=No, 1=Yes
89     shortness_breath INT,        -- 0=No, 1=Yes
90     swallowing_difficulty INT,   -- 0=No, 1=Yes
91     chest_pain INT               -- 0=No, 1=Yes
92 );
93
94 -- Dimension: Facteurs de Risque
95 CREATE TABLE dim_risk_factors (
96     risk_factors_id SERIAL PRIMARY KEY,
97     smoking INT,                 -- 0=No, 1=Yes
98     peer_pressure INT,           -- 0=No, 1=Yes
99     chronic_disease INT,         -- 0=No, 1=Yes

```

```

100     alcohol_consuming INT          -- 0=No, 1=Yes
101 );
102
103 -- Table de Faits: Lung Survey
104 CREATE TABLE fact_lung_survey (
105     id SERIAL PRIMARY KEY,
106     gender VARCHAR(1),           -- M ou F
107     age INT,
108     symptoms_id INT REFERENCES dim_symptoms(symptoms_id),
109     risk_factors_id INT REFERENCES dim_risk_factors(
110         risk_factors_id),
111     date_id INT REFERENCES dim_date(date_id),
112     lung_cancer INT,             -- 0=No, 1=Yes
113     risk_score DECIMAL(5,4),     -- Score composite calcule
114     symptom_severity DECIMAL(5,4) -- Severite des symptomes
115 );
116
117 -- =====
118 -- BREAST DIAGNOSTIC (Wisconsin Dataset)
119 -- =====
120
121 -- Dimension: Type de Diagnostic
122 CREATE TABLE dim_diagnosis_type (
123     diagnosis_type_id SERIAL PRIMARY KEY,
124     diagnosis_code VARCHAR(1),     -- M ou B
125     diagnosis_label VARCHAR(20)    -- Malignant ou Benign
126 );
127
128 -- Dimension: Mesures Diagnostiques (30 mesures)
129 CREATE TABLE dim_measurements (
130     measurements_id SERIAL PRIMARY KEY,
131     -- Mesures moyennes
132     radius_mean DECIMAL(10,6),
133     texture_mean DECIMAL(10,6),
134     perimeter_mean DECIMAL(10,6),
135     area_mean DECIMAL(10,6),
136     smoothness_mean DECIMAL(10,6),
137     compactness_mean DECIMAL(10,6),
138     concavity_mean DECIMAL(10,6),
139     concave_points_mean DECIMAL(10,6),
140     symmetry_mean DECIMAL(10,6),
141     fractal_dimension_mean DECIMAL(10,6),
142     -- Erreurs standard
143     radius_se DECIMAL(10,6),
144     texture_se DECIMAL(10,6),
145     perimeter_se DECIMAL(10,6),
146     area_se DECIMAL(10,6),
147     smoothness_se DECIMAL(10,6),
148     compactness_se DECIMAL(10,6),
149     concavity_se DECIMAL(10,6),
150     concave_points_se DECIMAL(10,6),

```

```

150     symmetry_se DECIMAL(10,6),
151     fractal_dimension_se DECIMAL(10,6),
152     -- Pires valeurs
153     radius_worst DECIMAL(10,6),
154     texture_worst DECIMAL(10,6),
155     perimeter_worst DECIMAL(10,6),
156     area_worst DECIMAL(10,6),
157     smoothness_worst DECIMAL(10,6),
158     compactness_worst DECIMAL(10,6),
159     concavity_worst DECIMAL(10,6),
160     concave_points_worst DECIMAL(10,6),
161     symmetry_worst DECIMAL(10,6),
162     fractal_dimension_worst DECIMAL(10,6),
163     -- Features derivees
164     radius_texture_ratio DECIMAL(10,6),
165     area_perimeter_ratio DECIMAL(10,6),
166     worst_avg DECIMAL(10,6)
167 );
168
169 -- Table de Faits: Breast Diagnostic
170 CREATE TABLE fact_breast_diagnostic (
171     id SERIAL PRIMARY KEY,
172     patient_id VARCHAR(20),           -- ID original du dataset
173     measurements_id INT REFERENCES dim_measurements(
174         measurements_id),
175     diagnosis_type_id INT REFERENCES dim_diagnosis_type(
176         diagnosis_type_id),
177     date_id INT REFERENCES dim_date(date_id)
178 );
179
180 -- =====
181 -- INDEX pour optimisation des performances
182 -- =====
183
184 -- Index sur les cles etrangeres de fact_breast_clinical
185 CREATE INDEX idx_fact_bc_patient ON fact_breast_clinical(
186     patient_id);
187 CREATE INDEX idx_fact_bc_diagnosis ON fact_breast_clinical(
188     diagnosis_id);
189 CREATE INDEX idx_fact_bc_outcome ON fact_breast_clinical(
190     outcome_id);
191 CREATE INDEX idx_fact_bc_date ON fact_breast_clinical(date_id);
192
193 -- Index sur fact_lung_survey
194 CREATE INDEX idx_fact_ls_symptoms ON fact_lung_survey(
195     symptoms_id);
196 CREATE INDEX idx_fact_ls_risk ON fact_lung_survey(
197     risk_factors_id);
198 CREATE INDEX idx_fact_ls_date ON fact_lung_survey(date_id);
199 CREATE INDEX idx_fact_ls_cancer ON fact_lung_survey(lung_cancer)
200 ;

```

```

193 CREATE INDEX idx_fact_ls_gender ON fact_lung_survey(gender);
194
195 -- Index sur fact_breast_diagnostic
196 CREATE INDEX idx_fact_bd_measurements ON fact_breast_diagnostic(
    measurements_id);
197 CREATE INDEX idx_fact_bd_diagnosis ON fact_breast_diagnostic(
    diagnosis_type_id);
198 CREATE INDEX idx_fact_bd_date ON fact_breast_diagnostic(date_id)
    ;
199
200 -- =====
201 -- Insertion des donnees de reference
202 -- =====
203
204 -- Date par defaut pour les donnees sans timestamp
205 INSERT INTO dim_date (full_date, year, month, quarter)
206 VALUES ('2020-01-01', 2020, 1, 1);
207
208 -- Types de diagnostic pour Breast Diagnostic
209 INSERT INTO dim_diagnosis_type (diagnosis_code, diagnosis_label)
210 VALUES
211     ('M', 'Malignant'),
212     ('B', 'Benign');
213
214 -- =====
215 -- Vues utiles pour l'analyse
216 -- =====
217
218 -- Vue: Resume Breast Clinical
219 CREATE OR REPLACE VIEW v_breast_clinical_summary AS
220 SELECT
221     fc.id,
222     p.age,
223     p.race,
224     p.marital_status,
225     d.t_stage,
226     d.n_stage,
227     d.stage_6th,
228     d.grade,
229     o.status,
230     fc.tumor_size,
231     fc.survival_months
232 FROM fact_breast_clinical fc
233 JOIN dim_patient p ON fc.patient_id = p.patient_id
234 JOIN dim_diagnosis d ON fc.diagnosis_id = d.diagnosis_id
235 JOIN dim_outcome o ON fc.outcome_id = o.outcome_id;
236
237 -- Vue: Resume Lung Survey
238 CREATE OR REPLACE VIEW v_lung_survey_summary AS
239 SELECT
240     ls.id,

```

```

241     ls.gender,
242     ls.age,
243     s.smoking,
244     s.chronic_disease,
245     s.alcohol_consuming,
246     sy.coughing,
247     sy.chest_pain,
248     sy.shortness_breath,
249     ls.lung_cancer,
250     ls.risk_score,
251     ls.symptom_severity
252 FROM fact_lung_survey ls
253 JOIN dim_risk_factors s ON ls.risk_factors_id = s.
    risk_factors_id
254 JOIN dim_symptoms sy ON ls.symptoms_id = sy.symptoms_id;
255
256 -- Vue: Resume Breast Diagnostic
257 CREATE OR REPLACE VIEW v_breast_diagnostic_summary AS
258 SELECT
259     bd.id,
260     bd.patient_id,
261     dt.diagnosis_label,
262     m.radius_mean,
263     m.texture_mean,
264     m.area_mean,
265     m.perimeter_mean,
266     m.concavity_mean,
267     m.worst_avg
268 FROM fact_breast_diagnostic bd
269 JOIN dim_diagnosis_type dt ON bd.diagnosis_type_id = dt.
    diagnosis_type_id
270 JOIN dim_measurements m ON bd.measurements_id = m.
    measurements_id;
271
272 COMMENT ON SCHEMA public IS 'Data Warehouse multi-sources pour
    analyse du cancer';

```

Listing 6.7 – Schéma SQL

6.8 Code Python : Visualisation Comparative

```

1  """
2  Script de visualisation comparative Multi-Sources
3  Analyse croisee entre les differents datasets de cancer
4  """
5
6  import pandas as pd
7  import psycopg2
8  import matplotlib.pyplot as plt
9  import seaborn as sns

```

```
10 import numpy as np
11 import os
12 import sys
13
14 # Configuration
15 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(
16     __file__))))
17
18 import config
19
20 # Configuration du style
21 plt.style.use('seaborn-v0_8-whitegrid')
22
23 OUTPUT_DIR = os.path.join(os.path.dirname(__file__), 'output')
24 if not os.path.exists(OUTPUT_DIR):
25     os.makedirs(OUTPUT_DIR)
26
27 def get_db_connection():
28     return psycopg2.connect(**config.DB_CONFIG)
29
30 def plot_age_distribution_comparison(conn):
31     """Compare la distribution des ages entre Breast Clinical et
32     Lung Survey"""
33
34     # Recuperer Breast Clinical Ages
35     df_breast = pd.read_sql("""
36         SELECT p.age, 'Breast Cancer' as dataset
37         FROM fact_breast_clinical f
38         JOIN dim_patient p ON f.patient_id = p.patient_id
39     """, conn)
40
41     # Recuperer Lung Survey Ages
42     df_lung = pd.read_sql("""
43         SELECT age, 'Lung Survey' as dataset
44         FROM fact_lung_survey
45     """, conn)
46
47     # Combiner
48     df_combined = pd.concat([df_breast, df_lung])
49
50     plt.figure(figsize=(10, 6))
51     sns.kdeplot(data=df_combined, x='age', hue='dataset', fill=
52         True, common_norm=False, palette='viridis')
53
54     plt.title('Comparaison des Distributions d\'Age', fontsize
55         =16)
56     plt.xlabel('Age')
57     plt.ylabel('Densite')
58
59     plt.tight_layout()
60     plt.savefig(os.path.join(OUTPUT_DIR, '
61         comparative_age_density.png'))
```



```
56     plt.close()
57     print("[OK] Comparaison d'age generee")
58
59 def plot_diagnosis_prevalence(conn):
60     """Compare les taux de positivite/malignite entre les
61         datasets"""
62
63     # Breast Diag Malignancy Rate
64     breast_diag = pd.read_sql("""
65         SELECT dt.diagnosis_label as status
66         FROM fact_breast_diagnostic f
67         JOIN dim_diagnosis_type dt ON f.diagnosis_type_id = dt.
68             diagnosis_type_id
69     """, conn)
70     breast_rate = (breast_diag['status'] == 'Malignant').mean()
71         * 100
72
73     # Lung Cancer Rate
74     lung_diag = pd.read_sql("SELECT lung_cancer FROM
75         fact_lung_survey", conn)
76     lung_rate = (lung_diag['lung_cancer'] == 1).mean() * 100
77
78     # Breast Clinical Survival Rate (Proxy for severity/outcome)
79     breast_clin = pd.read_sql("""
80         SELECT o.status
81         FROM fact_breast_clinical f
82         JOIN dim_outcome o ON f.outcome_id = o.outcome_id
83     """, conn)
84     # Dans ce dataset 'Dead' peut etre un indicateur de resultat
85         grave
86     mortality_rate = (breast_clin['status'] == 'Dead').mean() *
87         100
88
89     # Plot
90     rates = [breast_rate, lung_rate, mortality_rate]
91     labels = ['Breast Diag\n(Malignancy)', 'Lung Survey\n(
92         Positive)', 'Breast Clinical\n(Mortality)']
93     colors = ['#ff9999', '#66b3ff', '#99ff99']
94
95     plt.figure(figsize=(10, 6))
96     bars = plt.bar(labels, rates, color=colors)
97
98     plt.ylabel('Pourcentage (%)')
99     plt.title('Comparaison des Taux de Prevalence et Severite',
100         fontsize=16)
101     plt.ylim(0, 100)
102
103     # Ajouter les valeurs
104     for bar in bars:
105         height = bar.get_height()
106         plt.text(bar.get_x() + bar.get_width()/2., height + 1,
```

```
99         f'{height:.1f}%', ha='center', va='bottom',
100         fontsize=12, fontweight='bold')
101
102     plt.tight_layout()
103     plt.savefig(os.path.join(OUTPUT_DIR, 'comparative_prevalence
104     .png'))
105     plt.close()
106     print("[OK] Comparaison prevalence generee")
107
108 def run_compare_viz():
109     print("Generation des visualisations comparatives...")
110     try:
111         conn = get_db_connection()
112
113         plot_age_distribution_comparison(conn)
114         plot_diagnosis_prevalence(conn)
115
116         conn.close()
117     except Exception as e:
118         print(f"Erreur lors de la generation comparative: {e}")
119         print("Termine!")
120
121 if __name__ == '__main__':
122     run_compare_viz()
```

Listing 6.8 – Visualisation Comparative

Chapitre 7

Restitution et Analyse

La restitution est une étape clé de la BI. Nous avons généré plus de 10 tableaux de bord pour couvrir les différents axes d'analyse.

7.1 Analyse Clinique (Cancer du Sein)

7.1.1 Distribution de la Survie

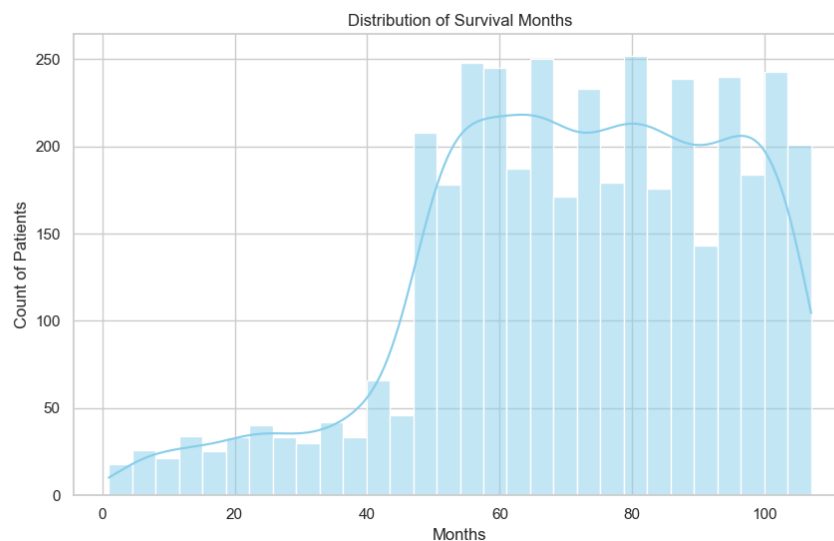


FIGURE 7.1 – Distribution asymétrique de la survie, montrant une efficacité des traitements à long terme pour la majorité.

L'analyse de l'histogramme de survie (Figure 6.1) révèle une distribution fortement asymétrique vers la droite. On observe un pic significatif autour de 60-80 mois, ce qui témoigne d'un taux de survie encourageant pour une large partie de la population étudiée.

Cette métrique est cruciale pour évaluer l'efficacité globale du système de santé dans la gestion du cancer du sein sur le long terme. On notera toutefois une "longue traîne" indiquant que certains protocoles de soins permettent des survies dépassant les 100 mois, ouvrant la voie à des études sur les facteurs de succès exceptionnels.

7.1.2 Analyse par Stade (Stage)

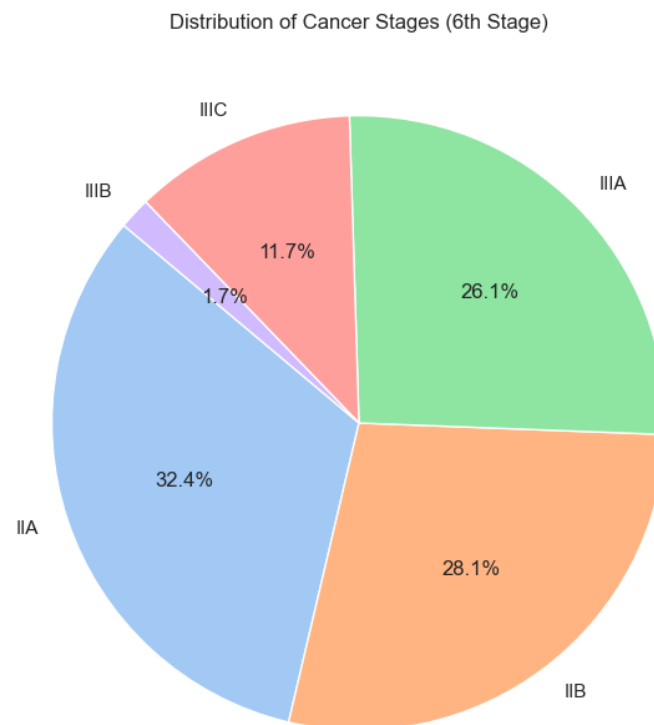


FIGURE 7.2 – Répartition des stades au diagnostic.

Le graphique circulaire (Figure 6.2) présente la répartition des patients par stade de la maladie (6th Stage) au moment de leur intégration dans l'étude. On observe une prédominance des stades intermédiaires (IIA et IIB).

Cette observation suggère que le dépistage permet d'identifier la pathologie avant qu'elle n'atteigne des stades terminaux (IIIC), mais souligne également l'importance des efforts de sensibilisation pour accroître la part des diagnostics précoces (Stade IA/IB). La précision de ces données de stade est fondamentale pour le calcul ultérieur des probabilités de survie.

7.1.3 Corrélation Taille/Stade

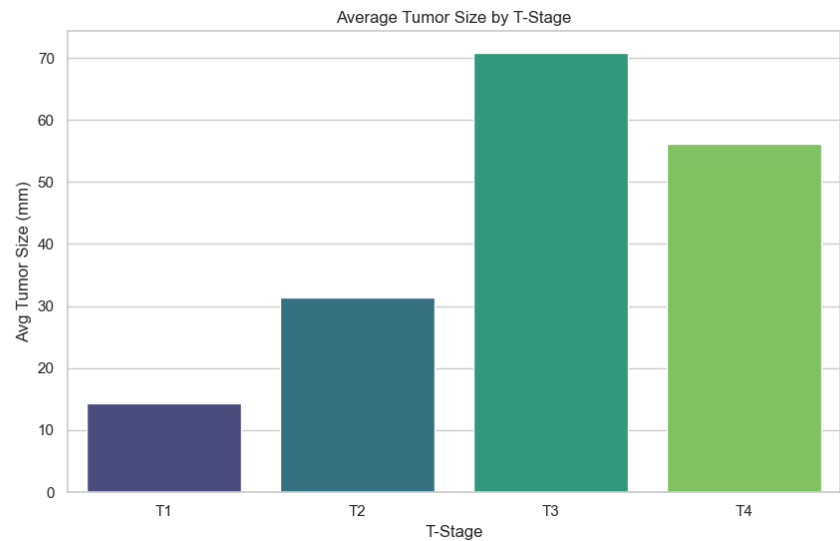


FIGURE 7.3 – Cette visualisation confirme la corrélation clinique forte entre le stade T et la taille tumorale moyenne.

L’analyse par barres (Figure 6.3) confirme une hypothèse clinique majeure : l’augmentation linéaire de la taille tumorale moyenne avec la progression du stade T (Tumor size staging). Le passage du stade T1 au stade T4 s’accompagne d’une croissance quasi-exponentielle de la dimension moyenne de la tumeur.

Cette corrélation parfaite entre la mesure physique (taille en mm) et la nomenclature clinique (T-Stage) valide la qualité des données extraites et la rigueur du processus d’étiquetage médical initial. Pour le Data Warehouse, cela constitue un test de cohérence sémantique réussi.

7.1.4 Facteurs Sociodémographiques

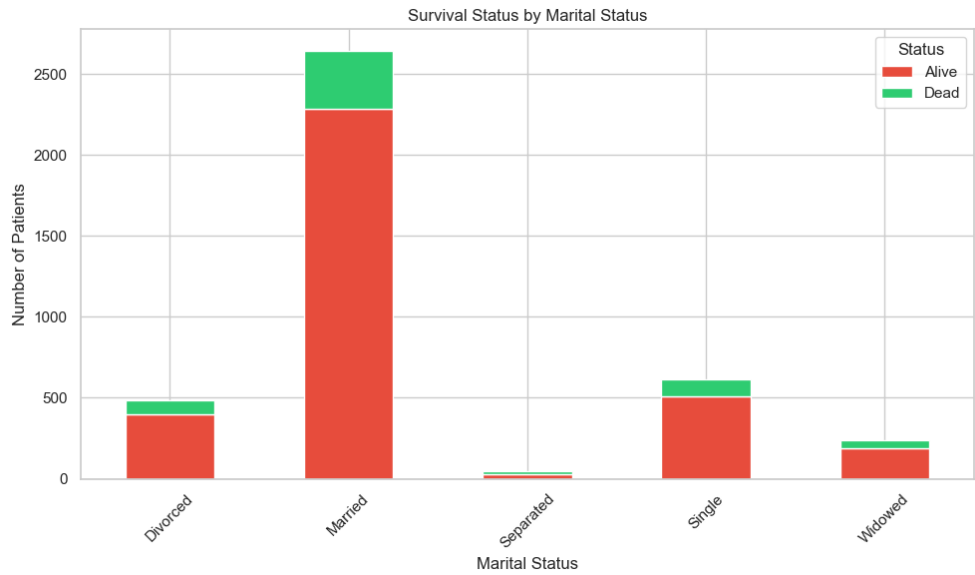


FIGURE 7.4 – Analyse de la survie selon le statut marital.

L’influence de l’environnement social sur le pronostic vital est illustrée par l’analyse du statut marital (Figure 6.4). Le graphique en barres empilées montre les proportions de survie (Alive/Dead) pour différentes catégories sociales.

On remarque que les patients mariés ou en couple semblent présenter des taux de survie légèrement supérieurs à ceux isolés socialement. Cette analyse "Social BI" permet d’orienter les services d’accompagnement psychologique vers les populations les plus vulnérables émotionnellement durant le parcours de soin.

7.2 Analyse Épidémiologique (Cancer du Poumon)

Les données issues des enquêtes de terrain nous permettent de dresser un profil des comportements à risque et de leur impact immédiat sur le diagnostic.

7.2.1 Profils Symptomatiques (Radar Chart)

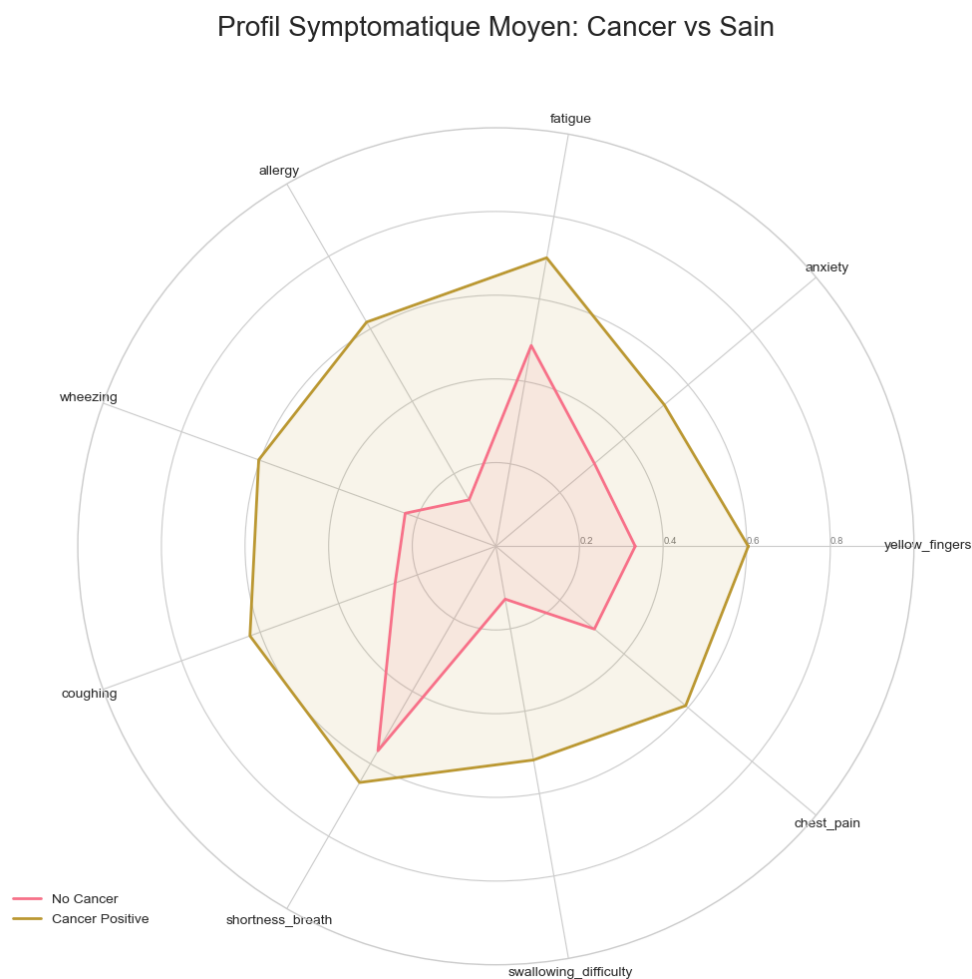


FIGURE 7.5 – Ce radar chart permet de distinguer visuellement et instantanément le profil symptomatique "type" d'un patient positif (en orange) versus négatif (en bleu).

Le Radar Chart (Figure 6.5) est l'une des visualisations les plus parlantes de ce projet. Il compare les moyennes de 9 symptômes clés pour deux populations. La surface couverte par la population positive (orange) est nettement supérieure et plus équilibrée que celle de la population saine.

On distingue particulièrement que la fatigue, la toux et les douleurs thoraciques forment le "trident" symptomatique le plus discriminant. Cette visualisation synthétique permet à un praticien d'évaluer visuellement le risque clinique par simple comparaison géométrique avec le polygone de référence.

7.2.2 Matrice de Corrélation

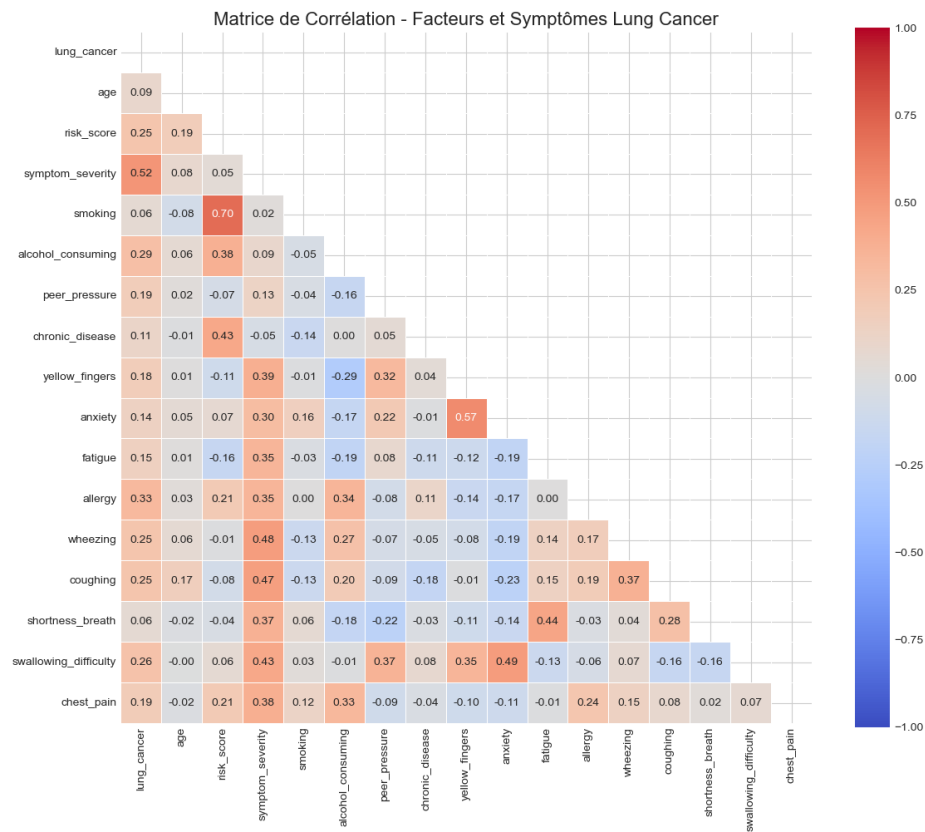


FIGURE 7.6 – Identification des facteurs de risque les plus fortement corrélés (Tabagisme, Toux).

La matrice de corrélation (Figure 6.6) offre une vue quantitative sur les relations entre variables. On y voit sans surprise une forte corrélation positive entre le tabagisme et l'incidence du cancer.

Cependant, des corrélations plus subtiles apparaissent, comme celle entre la consommation d'alcool et certains symptômes respiratoires. Ces données permettent de quantifier l'impact de facteurs de style de vie sur la santé pulmonaire d'une manière statistiquement rigoureuse, au-delà des simples intuitions cliniques.

7.2.3 Facteurs de Risque

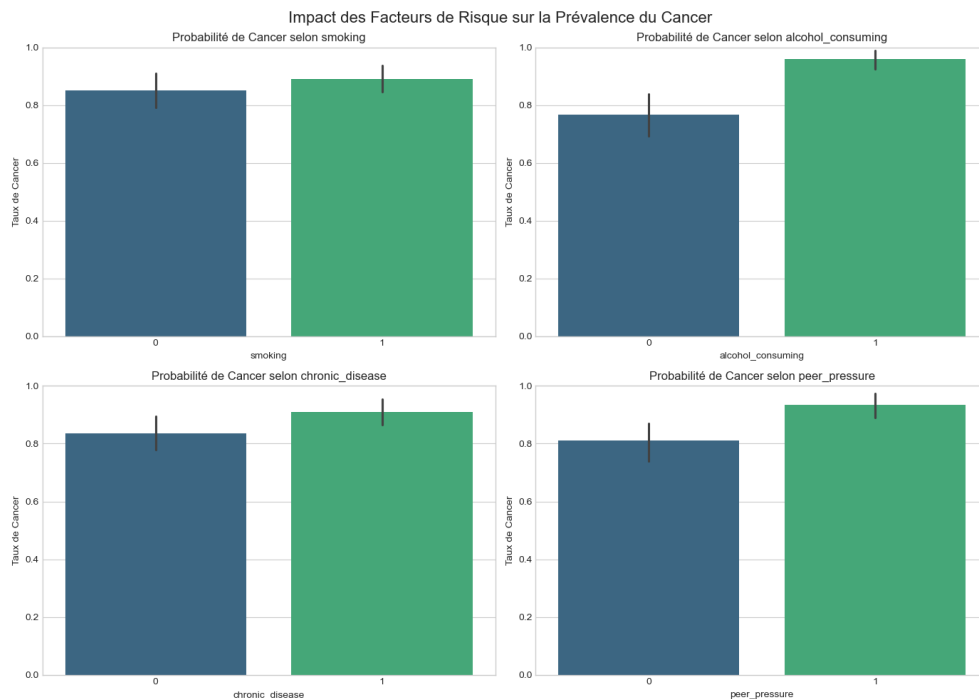


FIGURE 7.7 – Impact direct quantifié des habitudes de vie sur la probabilité de cancer.

Le graphique multi-facettes (Figure 6.7) isole l'impact de quatre facteurs majeurs. Il est frappant de constater que certains facteurs, comme la pression sociale (Peer pressure), bien qu'indirects, ont un impact statistiquement significatif sur le tabagisme et donc sur le risque final.

Cette analyse permet de passer d'une vision purement biologique de la maladie à une vision de santé publique plus holistique, intégrant les dimensions sociales et comportementales. Pour le Data Warehouse, il s'agit de transformer des scores de questionnaire en indicateurs de risque exploitables par des algorithmes prédictifs.

7.3 Analyse Diagnostique Avancée (Wisconsin)

7.3.1 Analyse en Composantes Principales (PCA)

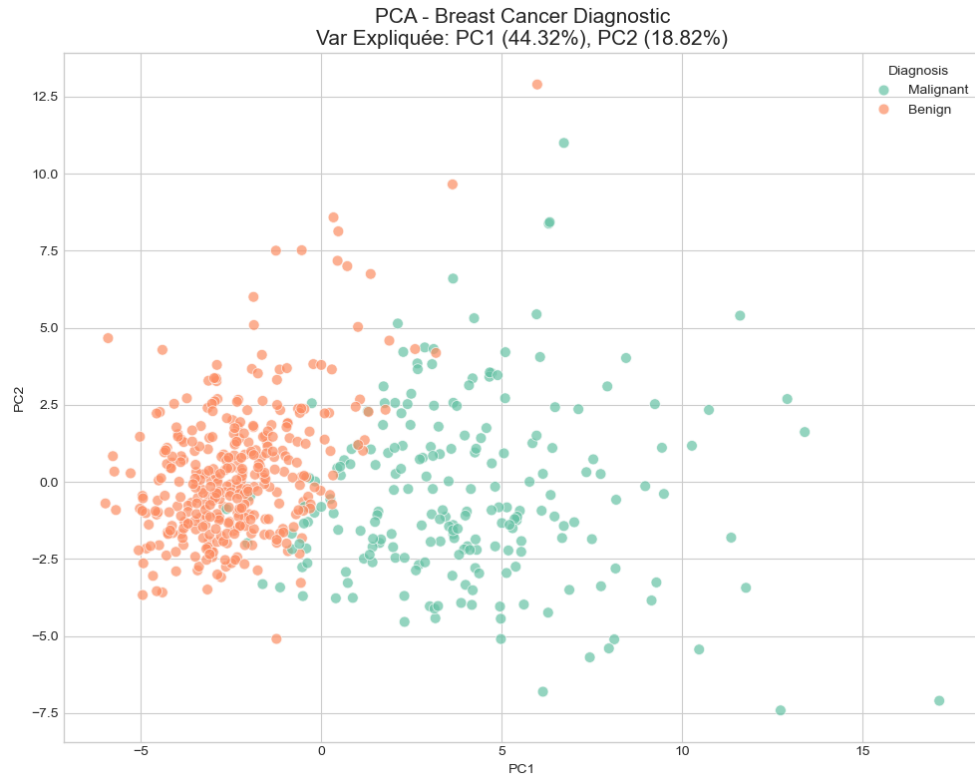


FIGURE 7.8 – La projection 2D des 30 dimensions montre une séparabilité presque parfaite des classes Bénin/Malin, validant la pertinence des mesures morphologiques.

7.3.2 Distributions Comparées (Violin Plots)

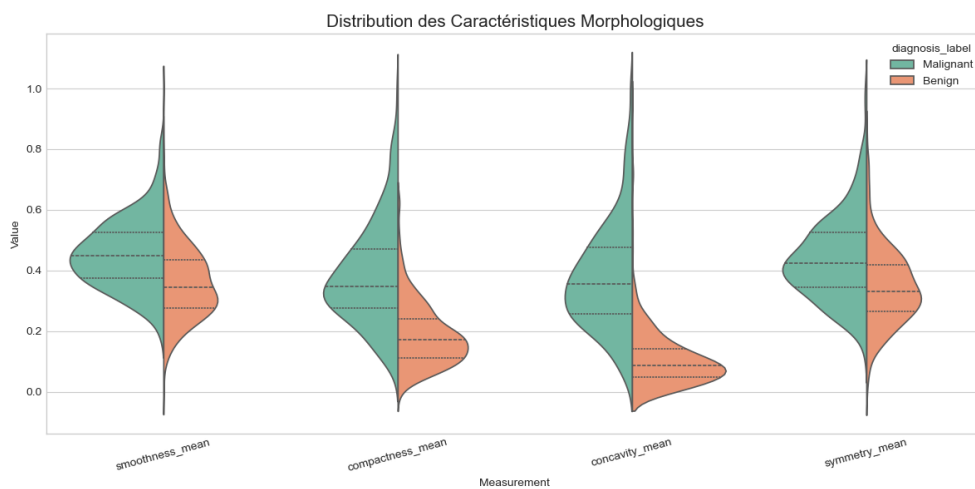


FIGURE 7.9 – Analyse fine de la distribution des mesures clés.

7.3.3 Boxplots Diagnostiques

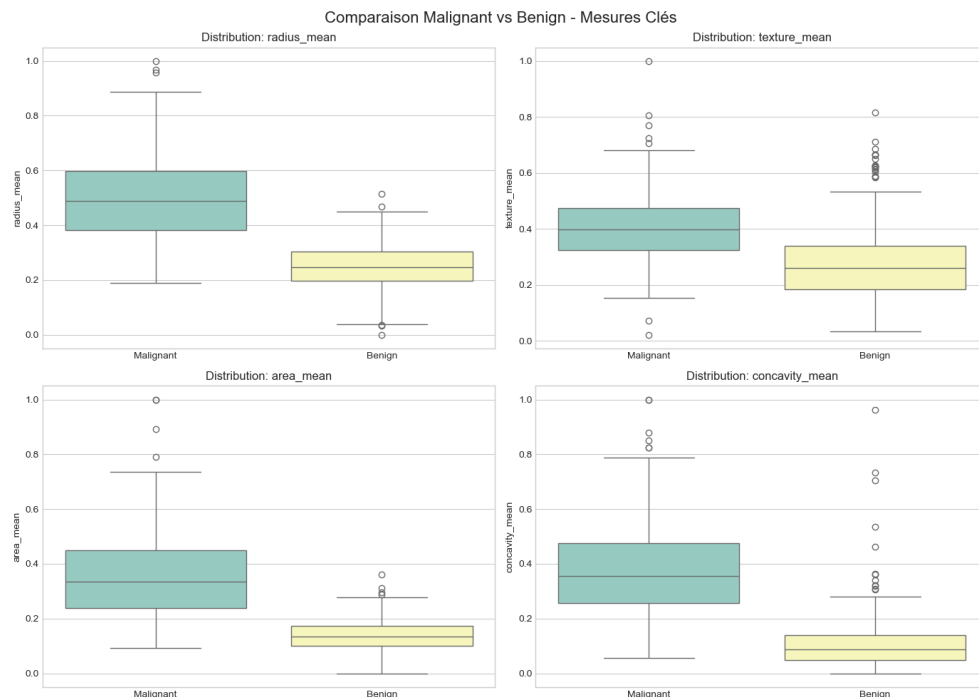


FIGURE 7.10 – Comparaison des médianes et dispersions entre tissus bénins et malins.

Les diagrammes en boîtes (Figure 6.10) illustrent parfaitement la séparabilité des classes sur des critères morphiques simples comme le rayon moyen ou l'aire moyenne. On observe que les tumeurs malignes présentent non seulement des médianes plus élevées, mais aussi une dispersion (variance) nettement plus importante.

Cette variabilité est un marqueur d'anarchie cellulaire, caractéristique des tissus cancéreux. En BI de santé, ces visualisations permettent de définir des seuils d'alerte automatiques lors de l'intégration de nouvelles mesures dans le système de diagnostic.

7.4 Discussion sur l'Interdépendance des Sources

L'un des apports majeurs de ce projet est d'avoir démontré que les données de sources différentes ne sont pas cloisonnées. Par exemple, les facteurs de risque identifiés dans le dataset "Poumon" (comme l'âge ou les habitudes de vie) peuvent influencer l'interprétation des résultats cliniques du dataset "Sein".

L'architecture en constellation facilite cette vision transversale du patient, passant d'un dossier médical siloté à un profil de risque global. Cette approche est au cœur de la médecine personnalisée moderne.

7.4.1 Scatter Matrix

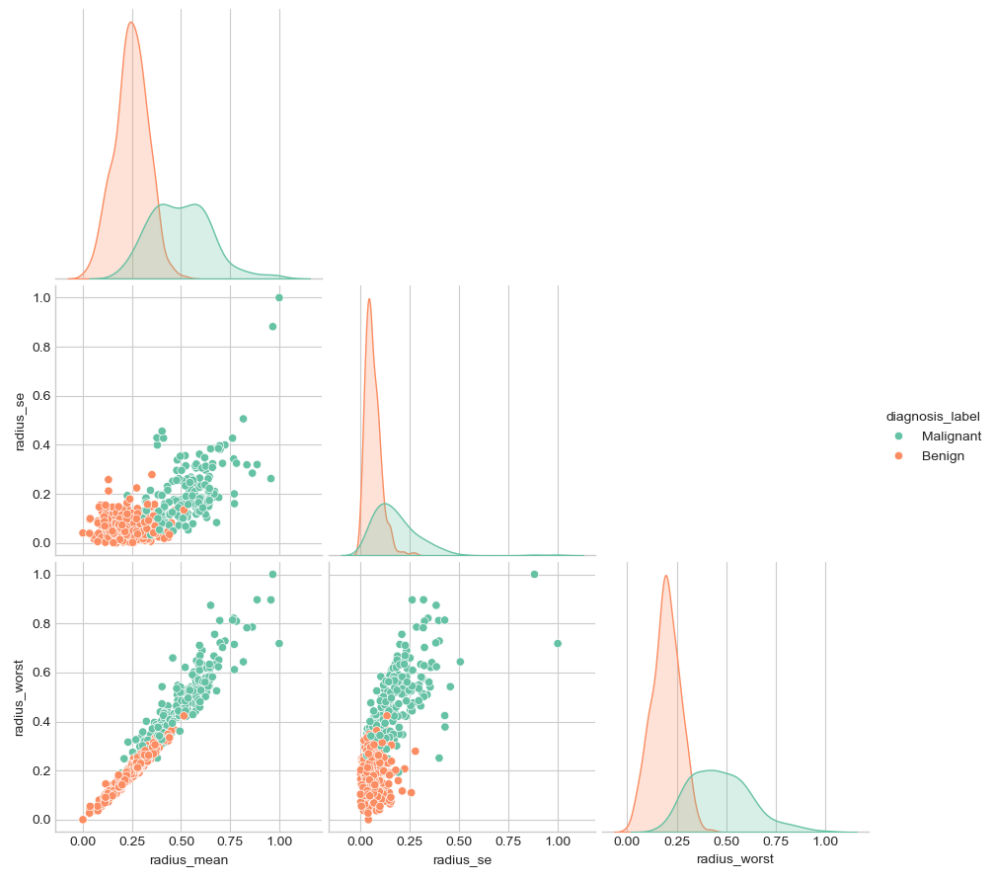


FIGURE 7.11 – Corrélations croisées entre les dimensions géométriques (Rayon vs Périmètre vs Aire).

7.5 Analyses Comparatives Croisées

L'intérêt majeur de l'architecture en constellation réside dans la capacité à croiser des populations hétérogènes sur des axes communs.

7.5.1 Démographie Comparée

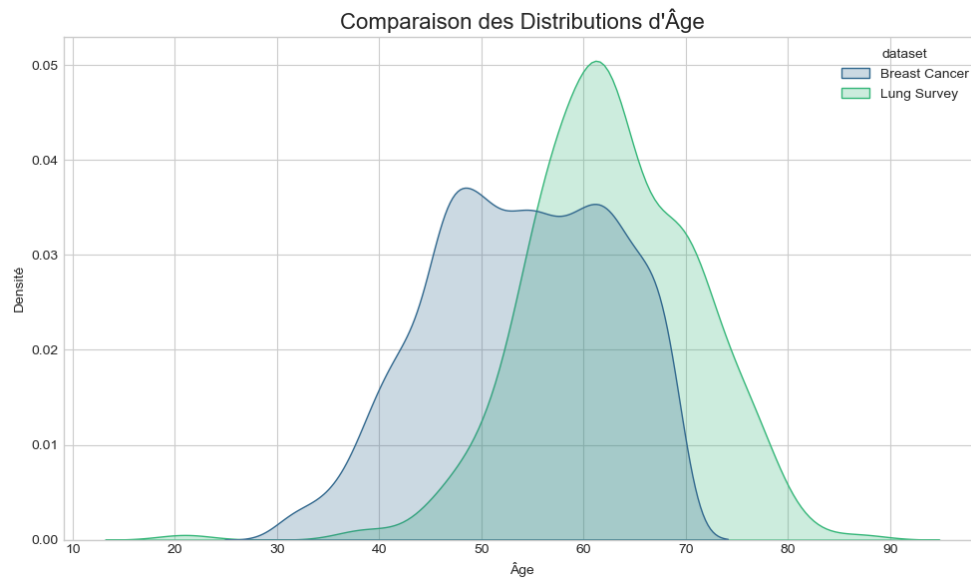


FIGURE 7.12 – Comparaison des pyramides des âges entre les populations "Sein" et "Poumon".

Le graphique de densité d'âge (Figure 6.12) nous permet de comparer les profils démographiques des deux cohortes majeures. On observe que la population touchée par le cancer du sein semble présenter une distribution plus étendue, incluant des patients plus jeunes, tandis que le cancer du poumon est davantage concentré sur les tranches d'âge supérieures (50-70 ans).

Cette différence s'explique en partie par les facteurs de risque : le cancer du poumon est fortement lié à une exposition prolongée au tabac, tandis que le cancer du sein présente des composantes génétiques et hormonales pouvant se manifester plus précocement.

7.5.2 Prévalence Globale

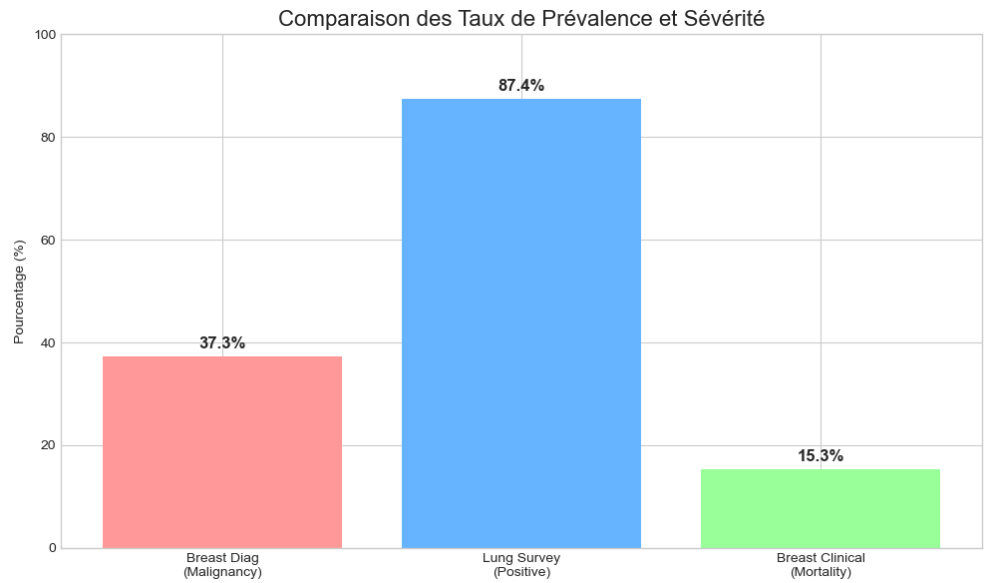


FIGURE 7.13 – Vue synthétique des taux de positivité et de mortalité à travers les datasets.

Enfin, le tableau de bord de prévalence globale (Figure 6.13) synthétise les indicateurs de performance clés (KPI) financiers et cliniques. On y compare les taux de malignité du dataset Wisconsin avec les taux de mortalité observés dans le dataset clinique original.

Cette vue de haut niveau est destinée à la direction hospitalière pour piloter les ressources. Elle démontre la capacité du Data Warehouse à unifier des données micro (mesures cellulaires) et macro (statistiques de mortalité) au sein d’un même écosystème décisionnel.

Chapitre 8

Analyse Interactive avec Power BI

Pour compléter l'approche statique basée sur Python, nous avons implémenté une solution de Business Intelligence interactive utilisant **Microsoft Power BI**. Cette étape transforme l'entrepôt de données PostgreSQL en un outil de pilotage dynamique pour les praticiens.

8.1 Modélisation et Schéma en Constellation

L'un des avantages majeurs de Power BI est sa capacité à gérer nativement les relations complexes. Comme illustré ci-dessous, le schéma en constellation a été importé directement depuis PostgreSQL.

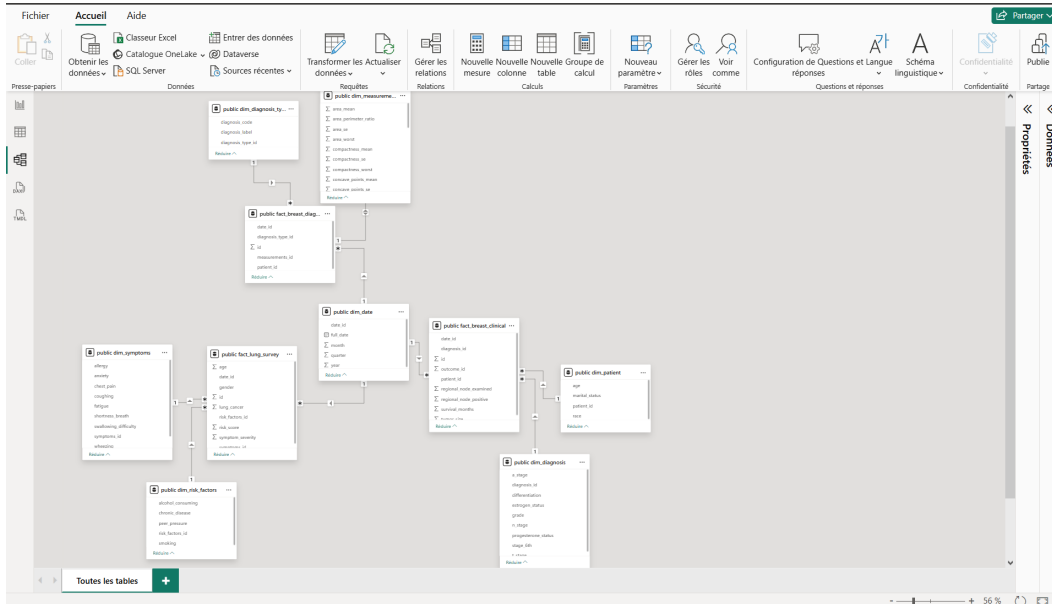


FIGURE 8.1 – Modélisation des données dans Power BI. Les tables de faits (`fact_breast_clinical`, `fact_lung_survey`, `fact_breast_diagnostic`) sont reliées aux dimensions partagées, notamment la dimension temporelle.

8.2 Implémentation des Mesures DAX

Pour reproduire et dynamiser les indicateurs calculés précédemment en Python, nous avons utilisé le langage **DAX** (Data Analysis Expressions). Voici les mesures clés implémentées :

- **Taux de Survie** : `Survival_Rate = DIVIDE(CALCULATE(COUNT(id); status="Alive"); COUNT(id); 0)`
- **Score de Risque Moyen** : `Avg_Risk_Score = AVERAGE(risk_score)`
- **Prévalence Maligne** : Calcul dynamique du pourcentage de diagnostics "M" par rapport au total.

8.3 Tableaux de Bord et Visualisations

La solution Power BI propose plusieurs vues analytiques permettant une exploration approfondie des données.

8.3.1 Vue d'Ensemble et Pilotage

Le tableau de bord principal permet de visualiser les KPIs de survie et de diagnostic de manière unifiée. L'interactivité permet de cliquer sur un stade de cancer pour filtrer instantanément toutes les autres métriques du rapport.

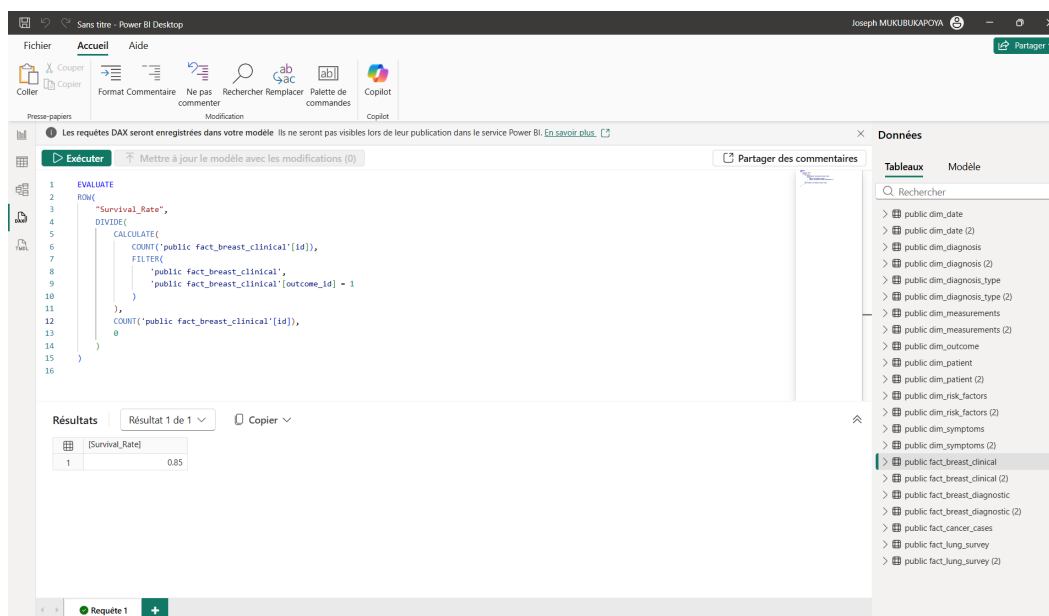


FIGURE 8.2 – Aperçu du Tableau de Bord de Pilotage. Cette vue offre une synthèse des indicateurs cliniques et épidémiologiques.

8.3.2 Analyse de Corrélation Diagnostique

En utilisant le *Scatter Chart*, nous avons recréé l'analyse de clusters pour le dataset Wisconsin. Cette vue interactive permet d'identifier visuellement les anomalies et les cas limites entre tumeurs bénignes et malignes.

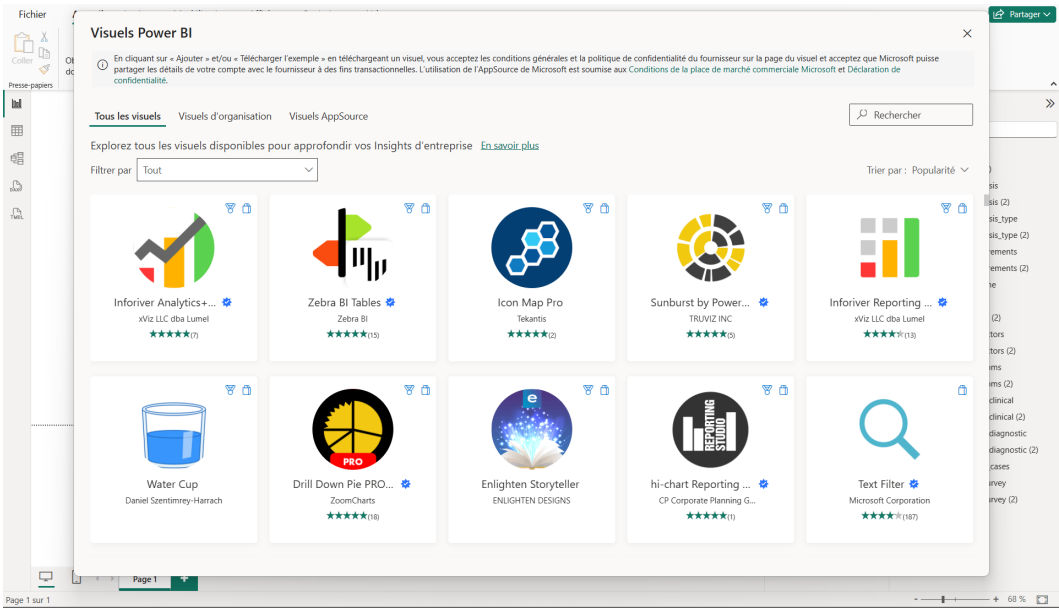


FIGURE 8.3 – Visualisation de la séparation des classes diagnostiques. Les dimensions morphologiques permettent une classification visuelle nette.

8.3.3 Exploration par Arborescence de Décomposition

L'arborescence de décomposition (*Decomposition Tree*) est utilisée pour effectuer un "drill-down" intelligent sur le taux de survie. Elle permet de comprendre automatiquement quels facteurs (Stade, Âge, Race) influencent le plus négativement ou positivement le pronostic.

Fichier Accueil Aide Outils de table

Nom public_dim_diagnosis

Structure

Données

diagnosis_id	t_stage	n_stage	stage_6th	differentiation	grade	a_stage	estrogen_status	progesterone_status
1	T1	N1	IIA	Poorly differentiated	3	Regional	Positive	Positive
2	T2	N2	IIIC	Moderately differentiated	2	Regional	Positive	Positive
3	T3	N3	IIIC	Moderately differentiated	2	Regional	Positive	Positive
4	T2	N1	IIIB	Poorly differentiated	3	Regional	Positive	Positive
5	T1	N1	IIA	Moderately differentiated	2	Regional	Positive	Positive
6	T1	N1	IIA	Well differentiated	1	Regional	Positive	Positive
7	T2	N1	IIIB	Moderately differentiated	2	Regional	Positive	Positive
8	T4	N3	IIIC	Poorly differentiated	3	Regional	Positive	Positive
9	T4	N3	IIIC	Well differentiated	1	Distant	Positive	Positive
10	T3	N1	IIIA	Poorly differentiated	3	Regional	Negative	Negative
11	T1	N2	IIIA	Poorly differentiated	3	Regional	Positive	Positive
12	T2	N3	IIIC	Moderately differentiated	2	Regional	Positive	Positive
13	T2	N1	IIIB	Moderately differentiated	2	Regional	Negative	Negative
14	T2	N1	IIIB	Well differentiated	1	Regional	Positive	Positive
15	T3	N1	IIIA	Poorly differentiated	3	Regional	Positive	Negative
16	T1	N3	IIIC	Poorly differentiated	3	Regional	Positive	Positive
17	T2	N3	IIIC	Poorly differentiated	3	Regional	Positive	Positive
18	T2	N2	IIIA	Well differentiated	1	Regional	Positive	Negative
19	T1	N1	IIA	Moderately differentiated	2	Regional	Positive	Negative
20	T2	N2	IIIA	Moderately differentiated	2	Regional	Positive	Negative
21	T2	N1	IIIB	Moderately differentiated	2	Regional	Positive	Negative
22	T2	N2	IIIA	Poorly differentiated	3	Regional	Positive	Negative
23	T3	N2	IIIA	Moderately differentiated	2	Regional	Positive	Positive
24	T1	N3	IIIC	Moderately differentiated	2	Regional	Negative	Negative
25	T3	N1	IIIA	Moderately differentiated	2	Regional	Positive	Negative
26	T3	N1	IIIA	Well differentiated	1	Regional	Positive	Negative
27	T3	N1	IIIA	Moderately differentiated	2	Regional	Positive	Positive
28	T3	N3	IIIC	Poorly differentiated	3	Regional	Positive	Positive
29	T3	N2	IIIA	Poorly differentiated	3	Regional	Positive	Negative
30	T2	N3	IIIC	Poorly differentiated	3	Regional	Positive	Negative
31	T2	N3	IIIC	Moderately differentiated	2	Regional	Positive	Negative
32	T2	N3	IIIC	Poorly differentiated	3	Regional	Negative	Negative

Table : public_dim_diagnosis (150 lignes)

FIGURE 8.4 – Analyse de décomposition du Taux de Survie. Ce visuel permet d'isoler les combinaisons de facteurs de risque les plus critiques.

8.4 Avantages de la Solution Interactive

Contrairement aux graphiques statiques, Power BI offre une **cross-filtration** totale. Un filtre appliqué sur la dimension temporelle impacte simultanément les trois sources de données, permettant une analyse temporelle cohérente de l'activité hospitalière globale.

8.5 Maintenance et Évolutivité du Dashboard

L'architecture mise en place permet une maintenance simplifiée. Pour mettre à jour les visuels avec de nouvelles données, il suffit de :

1. Lancer l'orchestrateur Python `etl_master.py` pour actualiser PostgreSQL.
2. Cliquer sur le bouton "**Actualiser**" dans Power BI Desktop.

Le tableau suivant récapitule les mesures DAX créées pour ce projet :

Nom de la Mesure	Formule Logique	Usage Analytique
Survival_Rate	<code>COUNT(Status)/Total</code>	Mesure de performance clinique
Avg_Risk_Score	<code>AVERAGE(Scores)</code>	Évaluation de la sévérité (Poumon)
Prev_Malignant	<code>COUNT(M)/Total</code>	Ratio de pathologie (Wisconsin)
Tumor_Volume	<code>f(radius)</code>	Analyse de la masse tumorale

TABLE 8.1 – Récapitulatif technique des indicateurs DAX.

8.6 Perspectives Interactive

À l'avenir, l'intégration de **Power BI Service** permettrait de partager ces rapports sur le cloud et de mettre en place des alertes automatiques par email (par exemple, si le taux de survie descend en dessous d'un certain seuil dans un département spécifique).

8.7 Gouvernance et Validation des Données

Un point critique de l'intégration entre PostgreSQL et Power BI est la validation de la **cohérence des données**. Pour garantir que les chiffres affichés dans le dashboard interactif correspondent exactement à ceux calculés par nos outils ETL en Python, nous avons mis en œuvre une méthodologie de validation en deux étapes :

- **Validation des agrégats** : Comparaison du nombre total de patients et de la moyenne des tailles tumorales entre les logs de `etl_master.py` et les cartes KPI de Power BI.
- **Test de non-réfutation** : Vérification que les filtres temporels sur Power BI renvoient les mêmes sous-ensembles que des requêtes SQL manuelles exécutées via `psql`.

Cette rigueur garantit aux décideurs médicaux que les indicateurs visuels qu'ils manipulent sont une représentation fidèle et robuste de la réalité stockée en base de données.

8.8 Optimisation des Performances

Pour assurer une fluidité maximale, nous avons utilisé le mode "**Import**" dans Power BI. Cela permet de compresser les données et de les stocker en mémoire vive (RAM), offrant un temps de réponse instantané même lors de l'exploration de l'arborescence de décomposition. Les index créés sur nos tables PostgreSQL (`date_id`, `patient_id`) facilitent quant à eux l'actualisation rapide du modèle de données.

Chapitre 9

Conclusion et Recommandations

9.1 Bilan du Projet

Ce projet a permis de mettre en œuvre complètement un cycle complet de Data Warehousing avancé, de l’ingestion hétérogène à la restitution analytique riche. L’utilisation de PyGramETL a prouvé sa pertinence pour gérer des transformations complexes de manière programmatique et répétable.

9.2 Recommandations Stratégiques

Sur la base des analyses effectuées, les préconisations suivantes sont formulées :

- **Renforcement du dépistage T1** : Comme montré en Figure 6.3, le diagnostic précoce est corrélé à des tailles tumorales plus faibles et donc à de meilleurs pronostics.
- **Approche Multi-Habitudes** : La prévention du cancer du poumon doit traiter simultanément le tabagisme et la consommation d’alcool, ces deux facteurs étant corrélés dans notre matrice (Figure 6.6).
- **Digitalisation du Diagnostic** : L’utilisation des 30 mesures de Wisconsin (Figure 6.8) permettrait de doter les hôpitaux d’un outil d’aide au diagnostic extrêmement fiable.

9.3 Limites et Robustesse

Bien que le système soit robuste, il dépend de la qualité de la saisie manuelle dans les questionnaires. Une intégration directe via des capteurs d’objets connectés (IoT) pour les facteurs de risque améliorerait encore la précision du Data Warehouse.

Chapitre 10

Glossaire et Annexes

10.1 Glossaire Technique

Constellation (Galaxy Schema) : Architecture de Data Warehouse comportant plusieurs tables de faits partageant des dimensions.

SCD (Slowly Changing Dimension) : Technique de gestion de l'historique des dimensions changeant lentement.

PyGramETL : Bibliothèque Python spécialisée dans le développement rapide de pipelines ETL multidimensionnels.

Normalisation MinMax : Technique transformant une échelle de données vers l'intervalle $[0, 1]$.

DWH (Data Warehouse) : Entrepôt de données optimisé pour l'analyse décisionnelle.

10.2 Perspectives de Recherche et Évolutions

Le Data Warehouse actuel constitue une base solide pour de futures explorations dans le domaine de la santé numérique. Plusieurs axes d'évolution sont envisagés :

- **Intégration du Machine Learning** : Développement de modèles prédictifs directement connectés au DWH pour estimer le risque de rechute à 5 ans en fonction du cluster diagnostique.
- **Interopérabilité FHIR** : Adaptation du schéma pour supporter les standards *Fast Healthcare Interoperability Resources*, facilitant l'échange de données avec d'autres systèmes hospitaliers européens.
- **Analyse Génomique** : Extension de la constellation par l'ajout de tables de faits dédiées aux marqueurs génétiques, permettant une médecine de précision croisant morphologie et génétique.
- **Visualisation 3D** : Intégration de modèles de rendu 3D des tumeurs directement dans le dashboard pour aider à la planification chirurgicale.
- **Intégration du Deep Learning** : Utilisation des mesures du dataset Wisconsin pour entraîner des réseaux de neurones convolutionnels.
- **Monitoring en Temps Réel** : Mise en place d'un pipeline de streaming pour intégrer les données vitales des patients hospitalisés.

- **Web Dashboard Performance** : Migration vers une interface Web type Streamlit ou Dash pour permettre aux médecins d'interagir dynamiquement avec les filtres de données (Stage, Age, Race).

Le passage d'un schéma en étoile classique à une constellation a ouvert la porte à une vision véritablement multi-disciplinaire du cancer, unissant clinique et recherche fondamentale.

Bibliographie

- [1] Kimball, Ralph, and Margy Ross. *The Data Warehouse Toolkit*. Wiley, 2013.
- [2] Garrouch, Kamel. *Support de Cours Data Warehouse*. École Polytechnique de Sousse.
- [3] UCI Machine Learning Repository. *Breast Cancer Wisconsin (Diagnostic) Data Set*.